

Eliciting causal models – Analysis

```
#options(device = svglite::svglite)
```

```
library(svglite)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.2.0      v readr      2.2.0
```

```
v forcats    1.0.1      v stringr    1.6.0
```

```
v ggplot2    4.0.2      v tibble     3.3.1
```

```
v lubridate  1.9.5      v tidyr      1.3.2
```

```
v purrr      1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(dagitty)
library(tidygraph)
```

Attaching package: 'tidygraph'

The following object is masked from 'package:dagitty':

convert

The following object is masked from 'package:stats':

filter

```
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

The following object is masked from 'package:readr':

col_factor

```
library(patchwork)
library(ggsci)
library(ggokabeito)
library(ggdist)
# library(ggpubfigs)
#library(cowplot)
```

Data import

Sessions

Graphical Causal Models

Modelling sessions were originally run on Loopy, then transcribed into DAGitty (online) to generate the following dot language syntax for importing into R. A better way of doing this is under development. Code hidden for this pdf render, can be inspected in the .qmd file.

Tidy graphs

Data imported as DAGitty objects, but need to be in tidygraph format for many calculations and visualisations.

```

# helper to get back door paths straight away - not working for all graphs yet
path_to_edges <- function(path) {
  tokens <- str_split(path, " ")[[1]]
  nodes <- tokens[c(TRUE, FALSE)]
  arrows <- tokens[c(FALSE, TRUE)]

  tibble(
    from = ifelse(arrows == ">", nodes[-length(nodes)], nodes[-1]),
    to = ifelse(arrows == ">", nodes[-1], nodes[-length(nodes)])
  )
}

open_backdoor_edgelist <- function(dag) {
  as_tibble(paths(backDoorGraph(dag), limit = 1e4)) |>
    filter(open) |>
    pull(paths) |>
    map_df(path_to_edges) |>
    distinct()
}

```

```

dag_to_tidy_graph <- function(dag){
  # Extract edges
  edges <- as.data.frame( dagitty::edges(dag) )
  edges$edge_w = 1
  nodes <- tibble(name = unique(c(edges$v, edges$w)))

  # Build tidygraph
  tg <- tbl_graph(nodes = nodes, edges = edges, directed = TRUE)
  return(tg)
}

dagitty_from_edge_df <- function(edges) {
  # edges is a df with "from" and "to"
  dagitty(str_c(
    "dag{",
    str_c(edges |>
      mutate(edge_string = str_c(from, ">", to)) |>
      pull(edge_string),
      collapse = ";\n")
    ,"}"))
}

```

Creating a list of all the DAGs in tidy format:

```
# making list of dagitty dags (used later)
ddags <- list(
  "S1" = dag_s1,
  "S2" = dag_s2,
  "S3" = dag_s3,
  "S4" = dag_s4,
  "S5" = dag_s5,
  "S6" = dag_s6,
  "S7" = dag_s7,
  "S8" = dag_s8
)

tdag_s1 <- dag_to_tidy_graph(dag_s1)
tdag_s2 <- dag_to_tidy_graph(dag_s2)
tdag_s3 <- dag_to_tidy_graph(dag_s3)
tdag_s4 <- dag_to_tidy_graph(dag_s4)
tdag_s5 <- dag_to_tidy_graph(dag_s5)
tdag_s6 <- dag_to_tidy_graph(dag_s6)
tdag_s7 <- dag_to_tidy_graph(dag_s7)
tdag_s8 <- dag_to_tidy_graph(dag_s8)

quick_list_of_all_nodes <- unique(c(tdag_s1 |> pull(name),
                                     tdag_s2 |> pull(name),
                                     tdag_s3 |> pull(name),
                                     tdag_s4 |> pull(name),
                                     tdag_s5 |> pull(name),
                                     tdag_s6 |> pull(name),
                                     tdag_s7 |> pull(name),
                                     tdag_s8 |> pull(name)
                                     )
)
```

Coding

Node Classifiers

Coding the nodes into rough groups. There are two levels:

- `node_group` is the first aggregation into slightly coarser groups of nodes.

- `node_type` is an even more coarse grouping into a handful of types.

Also some nicer names of the fields: - `node_nice_name` is a longer more explanatory name for plotting and putting in tables. - `node_group_nice_name` a longer more explanatory name for node groups, as above.

```
name_node_nice_name <- tribble(
  ~name, ~nice_name,
  "CD", "Any system data",
  "CD.Att", "Attendance",
  "CD.LMS", "LMS activity",
  "CD.LMS.code", "Code artifacts",
  "CD.LMS.write", "Writing artifacts",
  "CD.Str", "Course Data",
  "Grade", "Assessment grade",
  "Grade.Coms", "Mark - Communication",
  "Grade.Exam", "Mark - Exam",
  "Grade.Interp", "Mark - Interpretation",
  "Grade.Proc", "Mark - Learning process",
  "Grade.Quiz", "Mark - Quiz",
  "Grade.Ref", "Mark - Learning reflection",
  "Grade.Tech", "Mark - Technical",
  "H", "Help (general)",
  "H.AI", "Help from AI (any)",
  "H.AI.Agent", "Help from AI (agent)",
  "H.AI.evil", "Misuse of AI",
  "H.AI.evil.code", "Misuse of AI (code)",
  "H.AI.evil.write", "Misuse from AI (write)",
  "H.AI.Fb", "Help from AI (feedback)",
  "H.AI.good", "Help from AI (to learn)",
  "H.AI.Res", "Help from AI (research)",
  "H.AI.Study", "Help from AI (revision)",
  "H.Ment", "Help from mentor",
  "H.other.evil", "Cheating - non-AI",
  "H.Peer", "Help from peers",
  "H.Tut", "Help from teacher / tutor",
  "H.Tut.Fb", "Feedback from teacher / tutor",
  "LD", "Learning design",
  "LD.AIGuide", "Guidance on AI use",
  "LD.Task", "Task design",
  "LD.Task.Code", "Coding task design",
  "LD.Task.Grp", "Groupwork task design",
  "LD.Task.Other", "Other task design",
```

```

"LD.Task.Prac", "Practical task design",
"LD.Task.Write", "Writing task design",
"S.Act", "Student decisions / actions",
"S.Act.AIstrat", "Student AI strategy",
"S.Act.Fb", "Student acts on feedback",
"S.Act.Fb.Post", "Student acts on feedback between subjects",
"S.Act.Frm", "Student does formative tasks",
"S.Act.Perf", "Student performance on task",
"S.Act.Time", "Student commits time",
"S.Aff.Integ", "Student acting with integrity",
"S.Aff.Joy", "Student enjoyment",
"S.Aff.MindSet", "Student mindset",
"S.Aff.Safe", "Student feels safe",
"S.Aff.WrkLd", "Student workload pressure",
"S.Att.Eq", "Student equity group",
"S.Att.HSk", "Student seeks help",
"S.Att.Nro", "Student neuro-divergent",
"S.Att.SocCap", "Student social capital",
"S.Cp.Aca", "Student academic skill",
"S.Cp.EvJdg", "Student evaluative judgement",
"S.Cp.Fb", "Student feedback literacy",
"S.Cp.Grp", "Student groupwork capability",
"S.Cp.Ind", "Student individual work capability",
"S.Cp.Lrn", "Student metacognitive learning capabilities",
"S.Cp.SRL", "Student SRL",
"S.Kn", "Student subject knowledge",
"S.Kn.Coms", "Student communications knowledge",
"S.Kn.Interp", "Student interpretation knowledge",
"S.Kn.Prior", "Student prior knowledge",
"S.Kn.Tech", "Student technical knowledge",
"T.Act.UseAI", "Teacher use of AI",
"T.Aff.Integ", "Teacher acts with integrity",
"T.Aff.WrkLd", "Teacher workload pressure",
"T.Cp.Inst", "Teacher instruction quality",
"T.Cp.KnStu", "Teacher knows students",
"T.Cp.Mark", "Teacher reliably marks",
"T.Kn.AI", "Teacher AI literacy",
"T.Kn.Ass", "Teacher assessment literacy",
"T.Kn.Sub", "Teacher subject knowledge"
)

node_group_nice_name <- tribble(

```

```

~node_group, ~nice_name,
"Assessments", "Preliminary assessments",
"CD.Att", "Student attendance",
"CD.LMS", "LMS activity",
"CD.Str", "Course Data",
"Grade.Outcome", "Assessment grade",
"H.AI.evil", "Misuse of AI",
"H.AI.good", "Help from AI",
"H.Cheat", "Cheating - non-AI",
"H.Peer", "Help from peers",
"H.Tut", "Help from teacher / tutor",
"LD", "Learning design",
"S.Act", "Student decisions / actions",
"S.Att", "Student attributes",
"S.Cp", "Student metacognitive skill",
"S.EM", "Student emotion / motivation",
"S.Kn", "Student knowledge",
"S.Kn.Prior", "Student prior knowledge",
"T.Act", "Teacher use of AI",
"T.Cp", "Teacher capabilities",
"T.EM", "Teacher emotion / motivation",
"T.Kn", "Teacher knowledge"
)

```

Coding node groups

```

code_node_group <- function(v) {
  case_when(
    # Course data - Str, LMS -----
    v == "CD" ~ "CD.Str", # was blending LMS and Str, but original graph was more about :
    str_detect(v, "CD\\.LMS") ~ "CD.LMS",
    v == "CD.Att" ~ "CD.Att",
    v == "CD.Str" ~ "CD.Str",
    # Grade data -----
    v == "Grade" ~ "Grade.Outcome",
    str_detect(v, "Grade\\.") ~ "Assessments",
    # Kinds of help -----
    # - - - Help from teacher or tutor
    v == "H" ~ "H.Tut", # closest match for the only broad category one
    v == "H.Tut" ~ "H.Tut",

```

```

v == "H.Ment" ~ "H.Tut",
v == "H.Tut.Fb" ~ "H.Tut",
# - - - Help from peers
v == "H.Peer" ~ "H.Peer",
# - - - Help from AI (good or bad usage)
str_detect(v, "H\\.AI\\.evil") ~ "H.AI.evil",
str_detect(v, "H\\.AI") ~ "H.AI.good",
# - - - Other nefarious help (i.e. cheating)
v == "H.other.evil" ~ "H.Cheat",
# Learning Design -----
str_detect(v, "^LD") ~ "LD",
# Student activity -----
str_detect(v, "^S\\.Act") ~ "S.Act",
# Student attributes (generally unchangable) ---
str_detect(v, "^S\\.Att") ~ "S.Att",
# Student metacognitive capability (metacognition)
str_detect(v, "^S\\.Cp") ~ "S.Cp",
# Student emotion / motivation
str_detect(v, "^S\\.Aff") ~ "S.EM",
# Student knowledge (cognition)
v == "S.Kn.Prior" ~ "S.Kn.Prior", # Prior knowledge treated as it's own node
str_detect(v, "^S\\.Kn") ~ "S.Kn",
# Teacher knowledge (cognition)
str_detect(v, "^T\\.Kn") ~ "T.Kn",
# Teacher capabilities (metacognition)
str_detect(v, "^T\\.Cp") ~ "T.Cp",
# Teacher emotion / motivation
str_detect(v, "^T\\.Aff") ~ "T.EM",
# Teacher actions / decisions
str_detect(v, "^T\\.Act") ~ "T.Act",
TRUE ~ v

)
}

```

Coding node types

```

code_node_type <- function(n) {
  fl <- str_extract(n, "^.{1}")
  case_when(

```

```

    fl == "S" ~ "Student",
    fl == "T" ~ "Teacher",
    fl == "C" ~ "Learning data",
    fl == "H" ~ "Help",
    fl == "L" ~ "Learning design",
    fl == "G" ~ "Assessment data"
  ) |>
    factor(levels = c("Assessment data", "Learning data", "Learning design", "Teacher", "
}

```

Coding node measurability

From the node groups, describe how easy (or not) they are to measure. This is in three levels:

- Directly: Data is available or easy enough to get directly for the node
- Proxy: Node is not directly measurable, but with work a direct proxy for the node could be found.
- Intangible: Node is not currently measurable, and unlikely to have a direct proxy.

By “direct proxy” I mean one step away in the causal graph. So if X, Y is not measurably, but Z is then $X \rightarrow Z$ would mean that X is measurable by proxy. However if at best we can only justify $X \rightarrow Y \rightarrow Z$ then X is intangible.

```

code_node_measurability <- function(node) {
  case_when(
    # Student
    node == "S.Att.SocCap" ~ "PO", # check
    node == "S.Att.Nro" ~ "O", # check
    node == "S.Att.Eq" ~ "DW", # check
    node == "S.Att.HSk" ~ "L", # check
    node == "S.Act.AIstrat" ~ "L",
    node == "S.Act.Fb" ~ "L",
    node == "S.Act.Fb.Post" ~ "L",
    str_detect(node, "^S\\.Act") ~ "PO",
    str_detect(node, "^S\\.Kn") ~ "PO",
    str_detect(node, "^S") ~ "L", # Latent
    # Help
    node == "H.other.evil" ~ "L",
    node == "H.Peer" ~ "L",
    node == "H.AI.evil.code" ~ "PO", # Potentially observable
    node == "H.AI.evil.write" ~ "PO",
    node == "H.AI.evil.write" ~ "PO",

```

```

    str_detect(node, "^H\\.AI") ~ "PO",
    str_detect(node, "^H") ~ "L",
    # Assessment
    node == "Grade" ~ "DW",
    node == "Grade.Exam" ~ "DW",
    str_detect(node, "^Grade") ~ "O",
    # Learning data
    node == "CD.LMS" ~ "DW",
    str_detect(node, "^CD\\.LMS") ~ "O",
    node == "CD.Att" ~ "PO",
    node == "CD.Str" ~ "DW", # check
    node == "CD" ~ "PO", # check
    # Learning design
    str_detect(node, "^LD") ~ "PO",
    # Teacher
    node == "T.Act.UseAI" ~ "PO",
    node == "T.Aff.WrkLd" ~ "PO", # check this, not act with integ as L
    str_detect(node, "^T") ~ "L",
    node == "Assessments" ~ "DW"
  ) |>
    factor(levels = c("DW", "O", "PO", "L"))
}

```

Graph processing

Graph helper functions

Convention is that these all start with `g_` and accept a `tidygraph` object as input.

Graph manipulator functions

```

# exposures(dag_s1)
library(igraph)

```

Attaching package: 'igraph'

The following object is masked from 'package:tidygraph':

groups

The following object is masked from 'package:dagitty':

edges

The following objects are masked from 'package:lubridate':

%--%, union

The following objects are masked from 'package:dplyr':

as_data_frame, groups, union

The following objects are masked from 'package:purrr':

compose, simplify

The following object is masked from 'package:tidyr':

crossing

The following object is masked from 'package:tibble':

as_data_frame

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

```

library(dagitty)

g_edgelist <- function(g){
  # tidygraph to tibble of edges
  g %E>%
    as_tibble() %>%
    mutate(
      from = g %N>% pull(name) %>% .[from],
      to   = g %N>% pull(name) %>% .[to]
    ) %>%
    select(from, to)
}

g_merge_nodes <- function(g, regex_from, string_to, no.self.loops = TRUE) {
  # merges nodes in a graph by renaming edgelist
  g_el <- g_edgelist(g)
  g_el_m <- g_el |>
    mutate(from = str_replace(from, regex_from, string_to),
           to = str_replace(to, regex_from, string_to)) |>
    distinct()
  tbl_graph(edges = g_el_m)
}

# same but for dags
dag_merge_nodes <- function(dag, mapping) {
  # mapping is a named list, like: c("Old.var" = "Merged.var", "Old.var2" = "Merged.var")
  g <- graph_from_data_frame(dagitty::edges(dag))
  old_names <- V(g)$name
  # Replace names using mapping if present, else keep original
  new_names <- ifelse(old_names %in% names(mapping),
                      mapping[old_names],
                      old_names)
  V(g)$name <- unname(new_names)
  new_g <- simplify(g, remove.loops = TRUE, remove.multiple = TRUE)

  # Convert back to dagitty
  edges_m <- igraph::as_data_frame(new_g, what="edges")
  dag_str <- paste0("dag { ", paste0(edges_m$from, " -> ", edges_m$to, collapse="; "), " }")
  g_m <- dagitty(dag_str)
  g_m
}

```



```

g_get_adjustment_sets <- function(g, exposure = "S.Kn", outcome = "Grade") {
  if (!is_acyclic(as.igraph(g))) {
    return(list("Cyclic" = "No adjustment sets"))
  }
  # Extract edges
  edges <- g_edgelist(g)

  # Build dagitty syntax
  edges_str <- edges %>%
    mutate(rel = paste0(from, " -> ", to)) %>%
    pull(rel)

  dagitty_str <- paste("dag {", paste(edges_str, collapse = "\n  "), "}", sep="\n")
  dagitty_obj <- dagitty(dagitty_str)

  # Compute adjustment sets
  sets <- adjustmentSets(dagitty_obj, exposure = exposure, outcome = outcome)
  as.list(sets)
}

```

Graph metrics functions

```

# would like pagerank with "Grade" excluded

g_node_metrics <- function(g) {
  pagerank_no_outcome <-
    g %N>%
    filter(name != "Grade") |>
    mutate(pagerank_no_outcome = centrality_pagerank(damping = 0.85)) |>
    as_tibble() |>
    select(name, pagerank_no_outcome)

  g %N>%
  mutate(
    node_w = 1,
    node_group = code_node_group(name),
    node_type = code_node_type(name),
    measurability = code_node_measurability(name),
    degree = centrality_degree(),
    btw = centrality_betweenness(),

```

```

    btw_rank = rank(-btw, ties.method = "min"), # rank (highest = 1)
    btw_rel_rank = (btw_rank - 1) / (n() - 1),
    btw_rel = 1 - btw_rel_rank,
    eigen = centrality_eigen(),
    pagerank = centrality_pagerank(damping = 0.5) # lowers the influence of sinks
  ) |>
  as_tibble() |>
  left_join(pagerank_no_outcome, by = "name") |>
  mutate(pagerank_no_outcome = replace_na(pagerank_no_outcome, 0))
}

```

```

g_summary_metrics <- function(g){
  pagerank_no_outcome <-
    g %N>%
    filter(name != "Grade") |>
    mutate(pagerank_no_outcome = centrality_pagerank(damping = 0.5)) |>
    as_tibble() |>
    select(name, pagerank_no_outcome)

  n.df <- g %N>%
  mutate(
    pgrnk = centrality_pagerank(damping = 0.5),
    ng = code_node_group(name),
    m = code_node_measurability(name)) |>
  as_tibble() |>
  left_join(pagerank_no_outcome, by = "name")

  # p_traced <- mean(n.df$m == "Traced")
  # p_untraced <- mean(n.df$m == "Untraced")
  # pgrnk_traced <- sum(n.df |> filter(m == "Traced") |> pull(pgrnk))
  # pgrnk_untraced <- sum(n.df |> filter(m == "Untraced") |> pull(pgrnk))
  # pgrnk_no_outcome_traced <- sum(n.df |> filter(m == "Traced") |> pull(pagerank_no_outcome))
  # pgrnk_no_outcome_untraced <- sum(n.df |> filter(m == "Untraced") |> pull(pagerank_no_outcome))

  p_dw <- mean(n.df$m == "DW")
  p_o <- mean(n.df$m == "O")
  p_po <- mean(n.df$m == "PO")
  p_l <- mean(n.df$m == "L")
  measure_metric <- (3 * p_dw + 2 * p_o + 1 * p_po) / 3

  graph_metrics <- list(
    order = gorder(g), # number of nodes

```

```

    size      = gsize(g), # number of edges
    density   = edge_density(g),
    diameter  = diameter(g), # longest shortest path
    mean_path = mean_distance(g),
    clustering= transitivity(g, type = "global"),
    acyclic   = igraph::is_acyclic(as.igraph(g)),
    measurability = measure_metric,
    p_dw = p_dw,
    p_o = p_o,
    p_po = p_po,
    p_l = p_l
  )
  graph_metrics
}

```

Generating a range of edge metrics - comparisons across models. Want to label edges in the unified DAG as 'contested' if they switch direction between some models.

```

tdags <- list(tdag_s1, tdag_s2, tdag_s3, tdag_s4,
             tdag_s5, tdag_s6, tdag_s7, tdag_s8)

g_labelled_edgelist <- function(g) {
  m <- as.character(substitute(g)) |>
    str_remove("^tdag_")
  g_edgelist(g) |>
    mutate(Model = m) |>
    select(Model, everything())
}

g_labelled_nodelist <- function(g) {
  m <- as.character(substitute(g)) |>
    str_remove("^tdag_")
  g %>%
    as_tibble() |>
    select(name) |>
    mutate(Model = m) |>
    select(Model, everything())
}

g_possible_edges <- function(g) {
  m <- as.character(substitute(g)) |>
    str_remove("^tdag_")
}

```

```

    g_labelled_nodelist(g) |>
      rename(from = name) |>
      mutate(Model = m) |>
      cross_join(g_labelled_nodelist(g) |>
        select(-Model) |>
        rename(to = name))
  }

all_possible_edges <- bind_rows(
  g_possible_edges(tdag_s1),
  g_possible_edges(tdag_s2)) |>
  bind_rows(g_possible_edges(tdag_s3)) |>
  bind_rows(g_possible_edges(tdag_s4)) |>
  bind_rows(g_possible_edges(tdag_s5)) |>
  bind_rows(g_possible_edges(tdag_s6)) |>
  bind_rows(g_possible_edges(tdag_s7)) |>
  bind_rows(g_possible_edges(tdag_s8))

max_possible_edges <- all_possible_edges |>
  filter(from != to) |>
  count(from, to) |>
  rename(max_n = n)

all_nodes <- bind_rows(g_labelled_nodelist(tdag_s1),
  g_labelled_nodelist(tdag_s2)) |>
  bind_rows(g_labelled_nodelist(tdag_s3)) |>
  bind_rows(g_labelled_nodelist(tdag_s4)) |>
  bind_rows(g_labelled_nodelist(tdag_s5)) |>
  bind_rows(g_labelled_nodelist(tdag_s6)) |>
  bind_rows(g_labelled_nodelist(tdag_s7)) |>
  bind_rows(g_labelled_nodelist(tdag_s8))

all_edges <- bind_rows(g_labelled_edgelist(tdag_s1),
  g_labelled_edgelist(tdag_s2)) |>
  bind_rows(g_labelled_edgelist(tdag_s3)) |>
  bind_rows(g_labelled_edgelist(tdag_s4)) |>
  bind_rows(g_labelled_edgelist(tdag_s5)) |>
  bind_rows(g_labelled_edgelist(tdag_s6)) |>
  bind_rows(g_labelled_edgelist(tdag_s7)) |>
  bind_rows(g_labelled_edgelist(tdag_s8))
reverses <- tibble(Model = all_edges$Model, from = all_edges$to, to = all_edges$from)

```

```

g_all_descendants_edge_list <- function(g, m) {
  # Get descendant edges
  expanded <- distances(g) < Inf
  closure_edges <- which(expanded, arr.ind = TRUE)

  expanded_edges <- data.frame(
    Model = m,
    from = V(g)[closure_edges[,1]]$name,
    to   = V(g)[closure_edges[,2]]$name
  ) |> dplyr::filter(from != to)
  return(expanded_edges)
}

all_descendants <-
  bind_rows(
    g_all_descendants_edge_list(tdag_s1, "s1"),
    g_all_descendants_edge_list(tdag_s2, "s2"),
  ) |>
  bind_rows(g_all_descendants_edge_list(tdag_s3, "s3")) |>
  bind_rows(g_all_descendants_edge_list(tdag_s4, "s4")) |>
  bind_rows(g_all_descendants_edge_list(tdag_s5, "s5")) |>
  bind_rows(g_all_descendants_edge_list(tdag_s6, "s6")) |>
  bind_rows(g_all_descendants_edge_list(tdag_s7, "s7")) |>
  bind_rows(g_all_descendants_edge_list(tdag_s8, "s8")) |>
  as_tibble()

all_edges_summary <-
  all_possible_edges |>
  filter(from != to) |>
  left_join(all_edges |> mutate(in_graph = TRUE)) |>
  mutate(in_graph = replace_na(in_graph, FALSE)) |>
  left_join(all_descendants |> mutate(descendant = TRUE)) |>
  mutate(descendant = replace_na(descendant, FALSE))

```

Joining with `by = join\_by(Model, from, to)`
 Joining with `by = join\_by(Model, from, to)`

```

all_edges_sum <-
  all_edges_summary |>
  group_by(from, to) |>

```

```

summarise(n = sum(in_graph),
          n_desc = sum(descendant)) |>
arrange(desc(n)) |>
ungroup()

```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by from and to.
- i Output is grouped by from.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(from, to))` for per-operation grouping
(`?dplyr::dplyr\_by`) instead.

```

all_edges_with_metrics <-
  all_edges_sum |>
  filter(n > 0) |>
  inner_join(max_possible_edges) |>
  mutate(
    edge_p_w = n / max_n,
    edge_dp_w = n_desc / max_n) |>
  mutate(edge_w = n)

```

Joining with `by = join\_by(from, to)`

```

# edge_not_chosen <-
#   all_possible_edges |>
#   inner_join(edges_with_metrics |>
#     filter(edge_p_w < 1) , relationship = "many-to-many") |>
#   anti_join(all_edges) |>
#   select(Model, from, to) |>
#   distinct() |>
#   left_join(
#     all_descendants |>
#       # anti_join(all_edges) |>
#       mutate(is_descendant = TRUE)) |>
#   mutate(is_descendant = replace_na(is_descendant, FALSE))

# edges_with_metrics <-
#   edges_with_metrics |>
#   left_join(edge_not_chosen |>
#     filter(is_descendant) |>

```

```
#           group_by(from, to) |>
#           summarise(n_mediated = sum(is_descendant))) |>
#   mutate(n_mediated = replace_na(n_mediated, 0)) |>
#   distinct()

switches <- inner_join(all_edges, reverses, by = c("from", "to"),
                       relationship = "many-to-many") |>
  arrange(Model.x)

edges_with_metrics <-
  all_edges_with_metrics |>
  left_join(switches |>
    select(from, to) |> mutate(switched = "contested")) |>
  mutate(switched = replace_na(switched, "aligned"))
```

Joining with `by = join\_by(from, to)`

```
edges_with_metrics |>
  count(switched) |>
  mutate(n = if_else(switched == "aligned", n, n/2)) |> # contested are counted twice due to
  mutate(p = n / sum(n))
```

```
# A tibble: 2 x 3
  switched      n      p
  <chr>    <dbl> <dbl>
1 aligned    181 0.968
2 contested     6 0.0321
```

```
edges_with_metrics |>
  filter(switched == "contested" | n > 1) |>
  count(switched) |>
  mutate(n = if_else(switched == "aligned", n, n/2)) |> # contested are counted twice due to
  mutate(p = n / sum(n))
```

```
# A tibble: 2 x 3
  switched      n      p
  <chr>    <dbl> <dbl>
1 aligned     7 0.538
2 contested     6 0.462
```

Also examining counts of the kinds of paths:

```
count_paths <- function(dag) {  
  tibble(  
    total      = nrow(as_tibble(paths(dag, limit = 1e4))),  
    backdoor   = nrow(as_tibble(paths(backDoorGraph(dag), limit = 1e4))),  
    frontdoor  = nrow(as_tibble(paths(dag, directed = TRUE, limit = 1e4)))  
  )  
}
```

Creating merged graph

```
library(ggraph)  
  
merged_edges <- g_edgelist(tdag_s1) |>  
  bind_rows(g_edgelist(tdag_s2)) |>  
  bind_rows(g_edgelist(tdag_s3)) |>  
  bind_rows(g_edgelist(tdag_s4)) |>  
  bind_rows(g_edgelist(tdag_s5)) |>  
  bind_rows(g_edgelist(tdag_s6)) |>  
  bind_rows(g_edgelist(tdag_s7)) |>  
  bind_rows(g_edgelist(tdag_s8)) |>  
  count(from, to) |> ungroup() |> rename(edge_w = n) |>  
  inner_join(edges_with_metrics)
```

Joining with `by = join\_by(from, to, edge\_w)`

```
merged_nodes <- tdag_s1 %N>% as_tibble() |>  
  bind_rows(tdag_s2 %N>% as_tibble()) |>  
  bind_rows(tdag_s3 %N>% as_tibble()) |>  
  bind_rows(tdag_s4 %N>% as_tibble()) |>  
  bind_rows(tdag_s5 %N>% as_tibble()) |>  
  bind_rows(tdag_s6 %N>% as_tibble()) |>  
  bind_rows(tdag_s7 %N>% as_tibble()) |>  
  bind_rows(tdag_s8 %N>% as_tibble()) |>  
  count(name) |> ungroup() |> rename(node_w = n) |>  
  mutate(node_group = code_node_group(name)) |>  
  mutate(node_type = code_node_type(name)) |>  
  mutate(measurable = code_node_measurability(name)) |>  
  left_join(name_node_nice_name, by = "name")
```



```
# cognitive, metacognitive, emotion / motivation, collaboration
merged_all_graph <- tbl_graph(nodes = merged_nodes, edges = merged_edges)
# View(merged_nodes |> arrange(name))
```

Coarsen graphs

This is based on the coding of the nodes into node groups.

```
g_coarsen_dag_by_ng <- function(g){
  # Creates a "coarser" graph by merging all nodes in the same node group
  old_nodes <- g %N% as_tibble()

  if (!("node_w" %in% names(old_nodes))) {
    old_nodes <- old_nodes |>
      mutate(node_w = 1)
  }

  new_nodes <- old_nodes |>
    mutate(measurability = code_node_measurability(name)) |>
    mutate(measurability_metric = case_when(
      measurability == "DW" ~ 1,
      measurability == "O" ~ 0.7,
      measurability == "PO" ~ 0.4,
      measurability == "L" ~ 0
    )) |>
    mutate(weighted_measurability = measurability_metric * node_w) |>
    mutate(name = code_node_group(name),
           node_type = code_node_type(name)) |>
    group_by(name, node_type) |>
    summarise(
      node_w = sum(node_w),
      measurability_metric = sum(weighted_measurability) / sum(node_w),
      .groups = "drop"
    )

  new_edges <- g |>
    g_edgelist() |>
    mutate(across(from:to, code_node_group)) |>
    count(from, to) |>
    rename(edge_w = n)
```

```
tbl_graph(nodes = new_nodes, edges = new_edges)
}
```

A more brutal coarsening by node type.

```
g_coarsen_dag_by_type <- function(g){
  new_nodes <- g %N>%
    as_tibble() |>
    mutate(name = code_node_type(name)) |>
    count(name) |>
    rename(node_w = n)

  new_edges <- g |>
    g_edgelist() |>
    mutate(across(from:to, code_node_type)) |>
    count(from, to) |>
    rename(edge_w = n)

  tbl_graph(nodes = new_nodes, edges = new_edges)
}
```

Here we merge nodes into node groups before generating plot later. This has the potential to likely introduce loops depending on how the constructs were built.

```
merged_coarse_models <-
  tbl_graph(
    nodes = all_nodes |>
      mutate(name = code_node_group(name)) |>
      distinct(Model, name) |>
      count(name) |>
      rename(node_w = n) |>
      inner_join(node_group_nice_name |> rename(name = node_group)),
    edges = all_edges |>
      mutate(across(from:to, code_node_group)) |>
      distinct() |>
      count(from, to) |>
      rename(edge_w = n)
  )
```

Joining with `by = join\_by(name)`

Graph metrics

Measurability, complexity

Model level metrics

```
all_paths_summary <- tibble(  
  Model = str_c("S", 1:8),  
  DAG   = list(dag_s1, dag_s2, dag_s3, dag_s4, dag_s5, dag_s6, dag_s7, dag_s8)  
) %>%  
  mutate(  
    counts = map(DAG, ~ count_paths(.x))  
  ) %>%  
  unnest(counts)  
  
all_paths <- tibble(  
  Model = str_c("S", 1:8),  
  Paths.df = list(  
    as_tibble(paths(dag_s1, limit = 5000)),  
    as_tibble(paths(dag_s2, limit = 5000)),  
    as_tibble(paths(dag_s3, limit = 5000)),  
    as_tibble(paths(dag_s4, limit = 5000)),  
    as_tibble(paths(dag_s5, limit = 5000)),  
    as_tibble(paths(dag_s6, limit = 5000)),  
    as_tibble(paths(dag_s7, limit = 5000)),  
    as_tibble(paths(dag_s8, limit = 5000))  
  )) |>  
  mutate(  
    n_paths = map_int(Paths.df, nrow),  
    n_open = map_int(Paths.df, ~sum(pull(.,open))),  
    p_open = map_dbl(Paths.df, ~mean(pull(.,open)))  
  )  
  
# Yes, this really should be implemented using map_df or something more elegant  
dag_complexity_metrics <-  
  bind_cols(tibble(Model = "S1"),  
    g_summary_metrics(tdag_s1) |>  
    as_tibble()) |>  
  bind_rows(  
    bind_cols(tibble(Model = "S2"),  
      g_summary_metrics(tdag_s2) |>
```

```

        as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S3"),
    g_summary_metrics(tdag_s3) |>
      as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S4"),
    g_summary_metrics(tdag_s4) |>
      as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S5"),
    g_summary_metrics(tdag_s5) |>
      as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S6"),
    g_summary_metrics(tdag_s6) |>
      as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S7"),
    g_summary_metrics(tdag_s7) |>
      as_tibble())
) |>
bind_rows(
  bind_cols(tibble(Model = "S8"),
    g_summary_metrics(tdag_s8) |>
      as_tibble())
) |>
inner_join(
  tibble(
    Model = str_c("S", 1:8),
    n_adj_sets = map_int(list(dag_s1, dag_s2, dag_s3, dag_s4,
      dag_s5, dag_s6, dag_s7, dag_s8),
      ~length(adjustmentSets(.)))
  ), by = "Model"
) |>
inner_join(all_paths_summary, by = "Model")

```

Graph dissimilarity

Taken from suggestions in [Grames et al. \(2022\)](#) this calculates the graph dissimilarity from [Shieber et al. \(2017\)](#).

```
source("d-measures.R") # Functions from Grames et al.
```

Creating distance matrices for both the fine and coarse graphs. The coarse seem to give a better ‘feel’ for the differences in the models. In the end a mixture distance is created using the average of the dissimilarity at each level: *node*, *group* and *type*.

```
coarsened_tdays <- map(tdays, g_coarsen_dag_by_ng)
coarsest_tdays <- map(tdays, g_coarsen_dag_by_type)

d_matrix_fine <-
  data.frame(
    S1 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s1)),
    S2 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s2)),
    S3 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s3)),
    S4 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s4)),
    S5 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s5)),
    S6 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s6)),
    S7 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s7)),
    S8 = map_dbl(tdays, ~calc_dissimilarity(., tdag_s8))
  )
```

Warning: `shortest.paths()` was deprecated in igraph 2.0.0.
i Please use `distances()` instead.

```
d_matrix_coarse <-
  data.frame(
    S1 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[1]])),
    S2 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[2]])),
    S3 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[3]])),
    S4 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[4]])),
    S5 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[5]])),
    S6 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[6]])),
    S7 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[7]])),
    S8 = map_dbl(coarsened_tdays, ~calc_dissimilarity(., coarsened_tdays[[8]]))
  )
```

```

d_matrix_coarsest <-
  data.frame(
    S1 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[1]])),
    S2 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[2]])),
    S3 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[3]])),
    S4 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[4]])),
    S5 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[5]])),
    S6 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[6]])),
    S7 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[7]])),
    S8 = map_dbl(coarsest_tdags, ~calc_dissimilarity(., coarsest_tdags[[8]]))
  )

d_matrix <- # trying coarse only - seems a better representation
  tibble(from = str_c("S", 1:8)) |>
  bind_cols(d_matrix_coarse)

d_matrix <- bind_cols( # weighted average: 3 parts coarse, 1 part fine. Fine alone seems to r
  tibble(from = str_c("S", 1:8)),
  (d_matrix_fine + d_matrix_coarse + d_matrix_coarsest) / 3)

d_edges <- d_matrix |>
  pivot_longer(cols = -from,
    names_to = "to") |>
  filter(value > 0) |>
  rename(edge_d = value) |>
  mutate(from = factor(from), to = factor(to)) |>
  filter(as.integer(from) < as.integer(to)) |>
  arrange(edge_d) |>
  mutate(edge_w = 1 / edge_d)

```

Node group level metrics

```

all_nodes_with_metrics <- g_node_metrics(tdag_s1) |>
  mutate(Model = "S1") |>
  bind_rows(
    g_node_metrics(tdag_s2) |>
      mutate(Model = "S2")
  ) |>
  bind_rows(

```

```

      g_node_metrics(tdag_s3) |>
        mutate(Model = "S3")
    ) |>
    bind_rows(
      g_node_metrics(tdag_s4) |>
        mutate(Model = "S4")
    ) |>
    bind_rows(
      g_node_metrics(tdag_s5) |>
        mutate(Model = "S5")
    ) |>
    bind_rows(
      g_node_metrics(tdag_s6) |>
        mutate(Model = "S6")
    ) |>
    bind_rows(
      g_node_metrics(tdag_s7) |>
        mutate(Model = "S7")
    ) |>
    bind_rows(
      g_node_metrics(tdag_s8) |>
        mutate(Model = "S8")
    )

```

Idea: can we get each nodes path length to a measurable (downstream / descendant) node?

```

g_min_distance_to_node <- function(g, node) {
  g %N>%
    mutate(dist_to = distances(as.igraph(.),
                               v = V(as.igraph(.)),
                               to = node,
                               mode = "out")[1,]) |>
    as_tibble()
}

g_distances <- function(g) {
  d <-
    distances(as.igraph(g), mode = "out")
  row_nodes <- rownames(d)
  bind_cols(tibble(from = row_nodes),
            as_tibble(d)) |>
    pivot_longer(-from, names_to = "to", values_to = "distance")
}

```

```

}

g_distancece_to_traced <- function(g) { #old
  g_distances(g) |>
    inner_join(all_nodes_with_metrics |>
      # filter(measurability == "Traced") |>
      filter(measurability == "DW") |>
      select(to = name) |>
      distinct()) |>
    group_by(from) |>
    summarise(dist_to_trace = min(distance)) |>
    rename(name = from)
}

all_model_nodes_distance_to_traced <-
  tibble(Model = "S1") |> bind_cols(g_distancece_to_traced(tdag_s1)) |>
  bind_rows(tibble(Model = "S2") |> bind_cols(g_distancece_to_traced(tdag_s2))) |>
  bind_rows(tibble(Model = "S3") |> bind_cols(g_distancece_to_traced(tdag_s3))) |>
  bind_rows(tibble(Model = "S4") |> bind_cols(g_distancece_to_traced(tdag_s4))) |>
  bind_rows(tibble(Model = "S5") |> bind_cols(g_distancece_to_traced(tdag_s5))) |>
  bind_rows(tibble(Model = "S6") |> bind_cols(g_distancece_to_traced(tdag_s6))) |>
  bind_rows(tibble(Model = "S7") |> bind_cols(g_distancece_to_traced(tdag_s7))) |>
  bind_rows(tibble(Model = "S8") |> bind_cols(g_distancece_to_traced(tdag_s8)))

```

```

Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`
Joining with `by = join_by(to)`

```

```

all_nodes_median_dist_to_traded <-
  all_model_nodes_distance_to_traced |>
  group_by(name) |>
  summarise(m_dist_to_trace = median(dist_to_trace))

all_node_groups_distance_to_traced <-
  all_model_nodes_distance_to_traced |>
  mutate(node_group = code_node_group(name)) |>

```



```
group_by(Model, node_group) |>
  summarise(m_dist_to_trace = mean(dist_to_trace))
```

`summarise()` has regrouped the output.
i Summaries were computed grouped by Model and node_group.
i Output is grouped by Model.
i Use `summarise(.groups = "drop\_last")` to silence this message.
i Use `summarise(.by = c(Model, node\_group))` for per-operation grouping
(`?dplyr::dplyr\_by`) instead.

Joining metrics and session data

```
gcm_metrics <-
  inner_join(session_data, dag_complexity_metrics, by = "Model")
```

Visualisations

Session and other

Figure: Time spent with model and modelling utility

```
# denser option - radial arc bars for observable
library(scales)
library(ggforce)
library(patchwork)
library(cowplot)
```

Attaching package: 'cowplot'

The following object is masked from 'package:patchwork':

align_plots

The following object is masked from 'package:lubridate':

stamp

```

hue1 <- palette.colors(3, palette = "Okabe-Ito")[[2]]
hue2 <- palette.colors(3, palette = "Okabe-Ito")[[3]]

# Main plot - hide all legends
p_validity_and_time <- gcm_metrics |>
  mutate(
    x_duration = as.numeric(Modelling.duration, units = "minutes"),
    y_validity = GCM.validity
  ) |>
  mutate(acyclic = if_else(acyclic, "Yes", "No")) |>
  ggplot(aes(
    color = acyclic,
    size = n_adj_sets
  )) +
  geom_point(aes(
    x = x_duration,
    y = y_validity)) +
  scale_y_continuous(
    limits = c(1, 4.05),
    breaks = c(1, 2, 3, 4),
    labels = c("Trivially", "Simply", "Adequately", "Well"),
    name = "Representation of GCM") +
  scale_x_continuous(
    limits = c(33, 52), # was 55 to Keep original limits for the data
    breaks = c(35, 40, 45, 50),
    name = "Time spent constructing model (minutes)") +
  scale_size(range = c(2, 7),
    breaks = range(gcm_metrics$n_adj_sets),
    name = "Adjustment\nsets (count)") +
  scale_color_manual(values = c(hue1, hue2), name = "Acyclic?") +
  scale_fill_manual(values = c(hue1, hue2), guide = "none") +
  theme_minimal() +
  # coord_fixed(ratio = 2
  #             # , clip = "off"
  #             ) +
  theme(
    legend.position = "left", # Hide legends from main plot
    # panel.grid.major.x = element_blank(), # Remove vertical gridlines
    panel.grid.minor = element_blank(),
    plot.margin = margin(10, 10, 10, 10)
  )

```

```

p_visibility <- gcm_metrics |>
  mutate(
    cp_dw = p_dw,
    cp_o = p_dw + p_o,
    cp_po = p_dw + p_o + p_po,
    cp_l = p_dw + p_o + p_po + p_l # should equal 1
  ) |>
  mutate(GCM.validity = case_when(
    GCM.validity == 3 & acyclic ~ GCM.validity - 0.03,
    GCM.validity == 3 & !acyclic ~ GCM.validity + 0.03,
    GCM.validity == 2.5 & acyclic ~ GCM.validity - 0.03,
    GCM.validity == 2.5 & !acyclic ~ GCM.validity + 0.03,
    TRUE ~ GCM.validity
  )) |>
  pivot_longer(c(starts_with("cp_"), starts_with("p_")), names_to = "node_vis", values_to = "value") |>
  mutate(node_visibility = case_when(
    str_detect(node_vis, "p_dw") ~ "Data Warehouse",
    str_detect(node_vis, "p_o") ~ "Observable",
    str_detect(node_vis, "p_po") ~ "Potentially observable",
    str_detect(node_vis, "p_l") ~ "Latent"
  )) |>
  ordered(levels = c(
    "Latent",
    "Potentially observable",
    "Observable",
    "Data Warehouse"
  ))) |>
  mutate(prop_type = if_else(str_detect(node_vis, "^cp_"), "cp", "p")) |>
  pivot_wider(names_from = prop_type, values_from = proportion, id_cols = c(Model,acyclic,
  mutate(acyclic = if_else(acyclic, "Yes", "No")) |>
  ggplot(aes(
    color = acyclic
  )) +
  geom_rect(aes(
    xmin = cp - p,
    xmax = cp,
    ymin = GCM.validity - 0.03,
    ymax = GCM.validity + 0.03,
    fill = acyclic,
    alpha = node_visibility),
    color = "white") +
  scale_y_continuous(

```

```

    limits = c(1, 4.05),
    breaks = c(1, 2, 3, 4),
    labels = c("Trivially", "Simply", "Adequately", "Well"),
    name = "Representation of GCM",
    position = "right") +
  scale_alpha_discrete(name = "Data visibility") +
  scale_fill_manual(values = c(hue1, hue2), guide = "none") +
  theme_minimal() +
  # coord_fixed(ratio = 0.15
  #             # , clip = "off"
  #             ) +
  xlab("Proportion of nodes") +
  theme(
    legend.position = "right", # Hide legends from main plot
    # panel.grid.major.x = element_blank(), # Remove vertical gridlines
    panel.grid.minor = element_blank(),
    plot.margin = margin(10, 10, 10, 10),
    axis.title.y = element_blank(),
    axis.text.y = element_blank()
  )

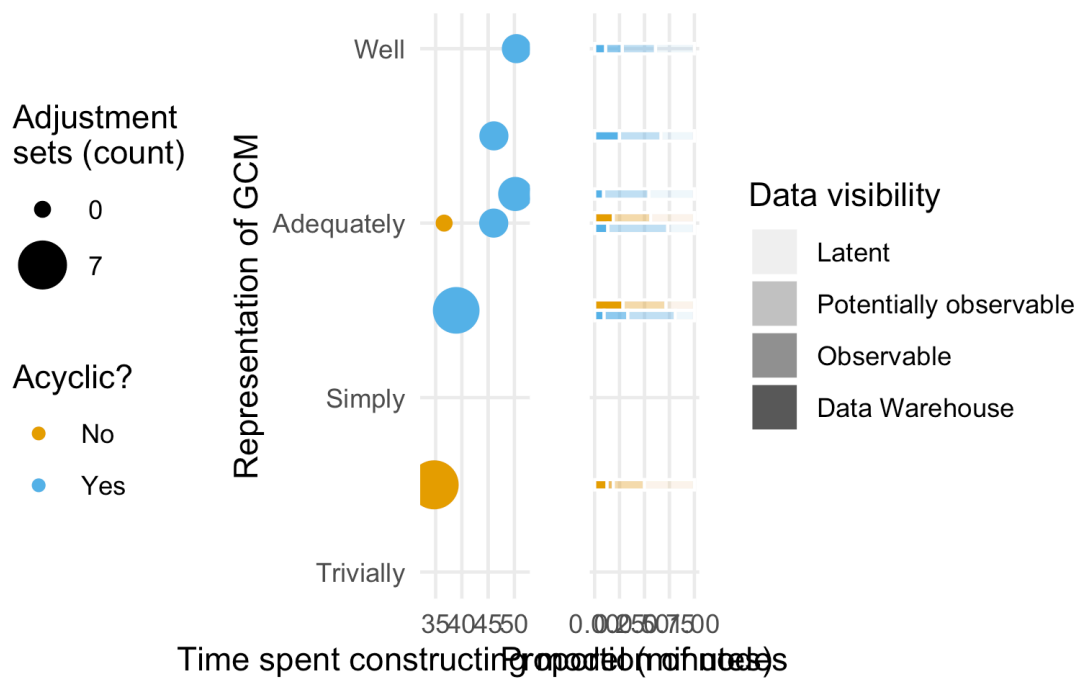
```

Warning: Using alpha for a discrete variable is not advised.

```

p_time_and_representation_1 <- p_validity_and_time + p_visibility
p_time_and_representation_1

```



```
ggsave("figures/FIG--time-modelling-and-representation.jpg",
  plot = p_time_and_representation_1,
  width = 10, height = 5,
  dpi = 300)

ggsave("figures/FIG--time-modelling-and-representation.svg",
  plot = p_time_and_representation_1,
  width = 25, height = 10, units = "cm")
```

This version displays all in one (is a little dense)

```
# denser option - radial arc bars for observable
library(scales)
library(ggforce)
library(patchwork)
library(cowplot)

hue1 <- palette.colors(3, palette = "Okabe-Ito")[[2]]
hue2 <- palette.colors(3, palette = "Okabe-Ito")[[3]]
```

```

# Main plot - hide all legends
p_main <- gcm_metrics |>
  mutate(
    x_duration = as.numeric(Modelling.duration, units = "minutes"),
    y_validity = GCM.validity * 10 + 20
  ) |>
  mutate(acyclic = if_else(acyclic, "Yes, DAG", "No, contained cycles")) |>
  ggplot(aes(
    color = acyclic,
    size = n_adj_sets
  )) +
  geom_arc_bar(aes(
    x0 = x_duration,
    y0 = y_validity,
    r0 = 0,
    r = 1.1,
    start = pi + 2 * (p_dw + p_o + p_po) * pi,
    end = 3 * pi
  ),
    fill = "white",
    linewidth = 0,
    alpha = 0.8, show.legend = FALSE) +
  geom_arc_bar(aes(
    x0 = x_duration,
    y0 = y_validity,
    fill = acyclic,
    size = 1,
    r0 = 0,
    r = 1.5,
    start = pi,
    end = pi + 2 * p_dw * pi
  ),
    linewidth = 0,
    alpha = 1, show.legend = FALSE) +
  geom_arc_bar(aes(
    x0 = x_duration,
    y0 = y_validity,
    fill = acyclic,
    r0 = 0,
    r = 1.3,
    start = pi + 2 * p_dw * pi,
    end = pi + 2 * (p_dw + p_o) * pi
  ),
    linewidth = 0,
    alpha = 1, show.legend = FALSE)

```

```

),
  linewidth = 0,
alpha = 0.6, show.legend = FALSE) +
geom_arc_bar(aes(
  x0 = x_duration,
  y0 = y_validity,
  fill = acyclic,
  r0 = 0,
  r = 1.1,
  start = pi + 2 * (p_dw + p_o) * pi,
  end = pi + 2 * (p_dw + p_o + p_po) * pi
),
  linewidth = 0,
alpha = 0.4, show.legend = FALSE) +
geom_point(aes(
  x = x_duration,
  y = y_validity)) +
scale_y_continuous(
  limits = c(30, 61),
  breaks = c(30, 40, 50, 60),
  labels = c("Trivially", "Simply", "Adequately", "Well"),
  name = "Representation of GCM") +
scale_x_continuous(
  limits = c(30, 55), # was 55 to Keep original limits for the data
  breaks = c(30, 40, 50),
  name = "Time spent constructing model (minutes)") +
scale_size(range = c(2, 7),
  breaks = range(gcm_metrics$n_adj_sets),
  name = "Adjustment\nsets") +
scale_color_manual(values = c(hue1, hue2), name = "Is DAG?") +
scale_fill_manual(values = c(hue1, hue2), guide = "none") +
theme_minimal() +
coord_fixed(ratio = 1, clip = "off") +
theme(
  legend.position = "none", # Hide legends from main plot
  # panel.grid.major.x = element_blank(), # Remove vertical gridlines
  panel.grid.minor = element_blank(),
  plot.margin = margin(10, 10, 10, 10)
)

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.

```

# Create a dummy plot just to extract the legends
p_for_legend <- gcm_metrics |>
  mutate(
    x_duration = as.numeric(Modelling.duration, units = "minutes"),
    y_validity = GCM.validity * 10 + 20
  ) |>
  mutate(acyclic = if_else(acyclic, "Yes, DAG", "No, contained cycles")) |>
  ggplot(aes(
    x = x_duration,
    y = y_validity,
    color = acyclic,
    size = n_adj_sets
  )) +
  geom_point() +
  scale_size(range = c(2, 7),
    breaks = range(gcm_metrics$n_adj_sets),
    name = "N adjustment sets") +
  scale_color_manual(values = c(hue1, hue2), name = "Is DAG?") +
  theme_minimal() +
  theme(legend.box = "vertical")

# Extract the legend
legend_extracted <- cowplot::get_legend(p_for_legend)

# Custom legend plot showing the arc explanation
p_legend_arcs <- ggplot() +
  geom_arc_bar(aes(x0 = 0, y0 = 0, r0 = 0.1, r = 0.8, start = pi, end = pi + pi/3),
    fill = "black", alpha = 1, linewidth = 0) +
  geom_arc_bar(aes(x0 = 0, y0 = 0, r0 = 0.1, r = 0.6, start = pi + pi/3, end = pi + (2*pi/3)),
    fill = "black", alpha = 0.6, linewidth = 0) +
  geom_arc_bar(aes(x0 = 0, y0 = 0, r0 = 0.1, r = 0.4, start = pi + (2*pi/3), end = 2 * pi),
    fill = "black", alpha = 0.4, linewidth = 0) +
  annotate("text",
    x = 0.5, y = -0.9,
    label = "In data\nwarehouse",
    hjust = 0, size = 3, color = "black") +
  annotate("text",
    x = 0.5, y = 0,
    label = "Observable",
    hjust = 0, size = 3, color = adjustcolor("black", alpha.f = 0.8)) +
  annotate("text", x = 0.5, y = 0.9, label = "Potentially\nobservable",
    hjust = 0, size = 3, color = adjustcolor("black", alpha.f = 0.6)) +

```



```

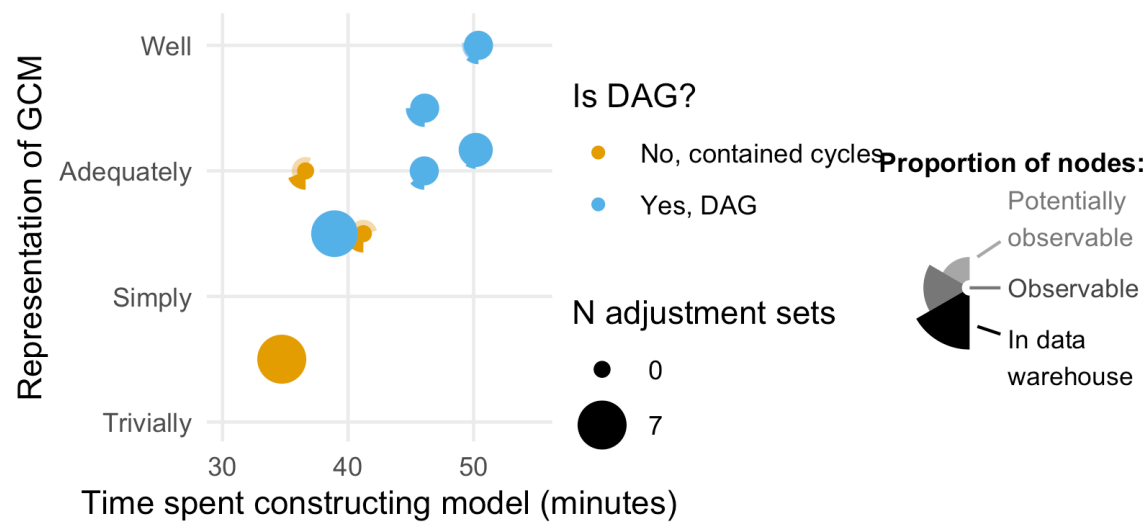
    annotate("segment", x = 0.1, xend = 0.4, y = -0.5, yend = -0.6, color = "black") +
    annotate("segment", x = 0, xend = 0.4, y = 0, yend = 0,
            color = "black", alpha = 0.6) +
    annotate("segment", x = 0.1, xend = 0.4, y = 0.4, yend = 0.6,
            color = "black", alpha = 0.4) +
    coord_fixed() +
    xlim(-1, 2.5) +
    ylim(-1.3, 1.3) +
    labs(title = "Proportion of nodes:") +
    theme_void() +
    theme(plot.title = element_text(size = 9, face = "bold"))

# Combine the arc legend and extracted legends vertically
legend_column <- cowplot::plot_grid(
  legend_extracted,
  p_legend_arcs,
  ncol = 2,
  rel_heights = c(1, 1)
)

# Combine main plot with legend column
p_time_and_representation <- cowplot::plot_grid(
  p_main,
  legend_column,
  ncol = 2,
  rel_widths = c(1, 1) # Adjust ratio as needed
)

p_time_and_representation

```



```
ggsave("figures/FIG--time-modelling-and-representation-arc-bars.jpg",
  plot = p_time_and_representation,
  width = 10, height = 5,
  dpi = 900)
```

As a table:

```
gcm_metrics |>
  mutate(acyclic = if_else(acyclic, "Yes, DAG", "No, contained cycles")) |>
  arrange(desc(Modelling.duration)) |>
  select(Modelling.duration, GCM.validity, acyclic, n_adj_sets, p_dw, p_o, p_po, p_l)
```

A tibble: 8 x 8

	Modelling.duration	GCM.validity	acyclic	n_adj_sets	p_dw	p_o	p_po	p_l
	<Period>	<dbl>	<chr>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
1	50M 23S	4	Yes, DAG	1	0.111	0.167	0.333	0.389
2	50M 10S	3.17	Yes, DAG	2	0.0909	0	0.455	0.455
3	46M 6S	3.5	Yes, DAG	1	0.25	0	0.417	0.333
4	46M 4S	3	Yes, DAG	1	0.133	0	0.6	0.267
5	41M 13S	2.5	No, cont~	0	0.286	0	0.429	0.286

6	38M	55S	2.5	Yes, DAG	6	0.0952	0.238	0.476	0.190
7	36M	37S	3	No, cont~	0	0.188	0	0.375	0.438
8	34M	43S	1.5	No, cont~	7	0.125	0.0625	0.312	0.5

Figure (OPTIONAL): Time spent with model and modelling utility - including graph similarity

Graph similarity computed with d-Measure.

Only showing the upper 50% of edges

```
d_between_models_median <- median(pull(d_edges, edge_d))

print(str_c("Median d-measure between models is ", round(d_between_models_median, 3)))
```

```
[1] "Median d-measure between models is 0.281"
```

```
d_edges |>
  slice(1)
```

```
# A tibble: 1 x 4
  from to   edge_d edge_w
<fct> <fct> <dbl> <dbl>
1 S3   S4     0.172  5.82
```

```
d_edges |>
  arrange(edge_w) |>
  slice(1)
```

```
# A tibble: 1 x 4
  from to   edge_d edge_w
<fct> <fct> <dbl> <dbl>
1 S2   S5     0.475  2.10
```

```
d_between_models_by_time_and_utility <-
  d_edges |>
  inner_join(
    gcm_metrics |>
      mutate(from = Model) |>
      select(from, xfrom = Modelling.duration, yfrom = GCM.validity)
```

```

) |>
inner_join(
  gcm_metrics |>
    mutate(to = Model) |>
    select(to, xto = Modelling.duration, yto = GCM.validity)
)

```

Joining with `by = join\_by(from)`

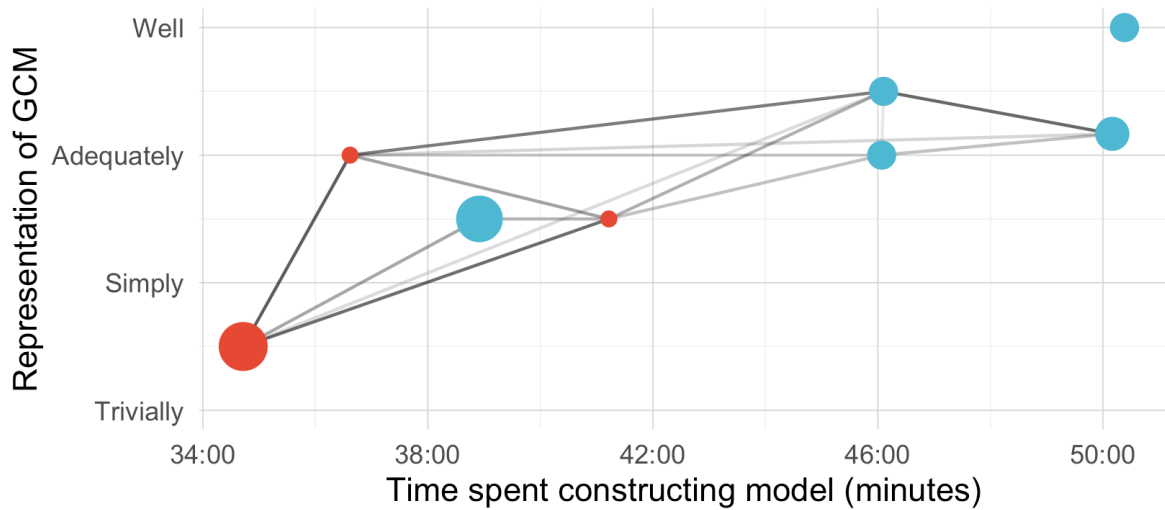
Joining with `by = join\_by(to)`

```

p_time_and_representaion_with_distance <-
  gcm_metrics |>
  mutate(acyclic = if_else(acyclic, "Yes, DAG", "No, contained cycles")) |>
  ggplot() +
  geom_segment(
    data = d_between_models_by_time_and_utility |>
      filter(edge_d < d_between_models_median),
    aes(x = xfrom, xend = xto, y = yfrom, yend = yto,
        alpha = edge_w), size = 0.5, color = "black") +
  geom_point(data = gcm_metrics,
             aes(x = Modelling.duration, y = GCM.validity, size = n_adj_sets, colour = acy
# geom_text(aes(label = Model), nudge_y = -.1) +
  scale_y_continuous(limits = c(1, 4),
                     breaks = c(1:4),
                     labels = c("Trivially", "Simply", "Adequately", "Well"),
                     name = "Representation of GCM") +
  scale_x_time(labels = label_time(format = "%M:%S"),
               name = "Time spent constructing model (minutes)") +
  scale_size(range = c(2,7),
             breaks = range(gcm_metrics$n_adj_sets),
             # labels = c("None", )
             name = "N adjustment sets") +
  scale_alpha_continuous(range = c(0.1, 0.7), guide = "none") +
  scale_color_npg(name = "Is DAG?") +
  theme_minimal() +
  theme(legend.position = "bottom",
        legend.direction = "vertical",
        panel.grid.major = element_line(color = "grey85", linewidth = 0.2),
        panel.grid.minor = element_line(color = "grey92", linewidth = 0.1)
  )

p_time_and_representaion_with_distance

```



N adjustment sets



Is DAG?



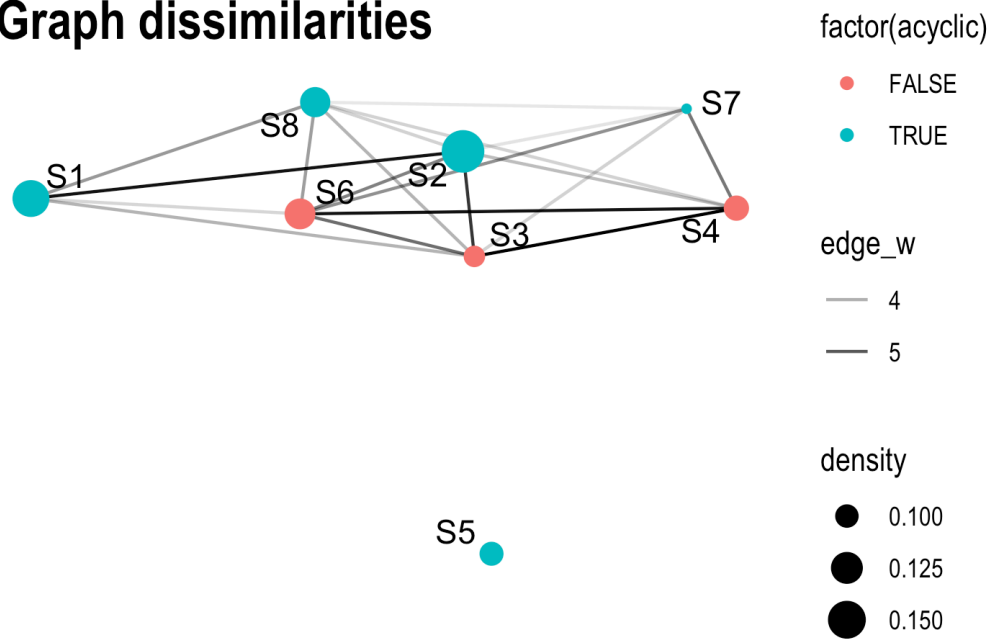
```
# ggsave("figures/FIG--time-modelling-and-representation-with-distances.jpg",
#       width = 5, height = 4,
#       dpi = 900)
```

DAG networks

Visualising model dissimilarities

```
tbl_graph(
  nodes = dag_complexity_metrics |> rename(name = Model),
  edges = d_edges) |>
  activate("edges") |>
  filter(edge_d < 0.33) |>
  ggraph(layout = "fr") +
  geom_edge_fan(aes(alpha = edge_w)) +
  geom_node_point(aes(size = density, color = factor(acyclic))) +
  geom_node_text(aes(label = name), repel = T) +
  theme_graph() +
  ggtitle("Graph dissimilarities")
```

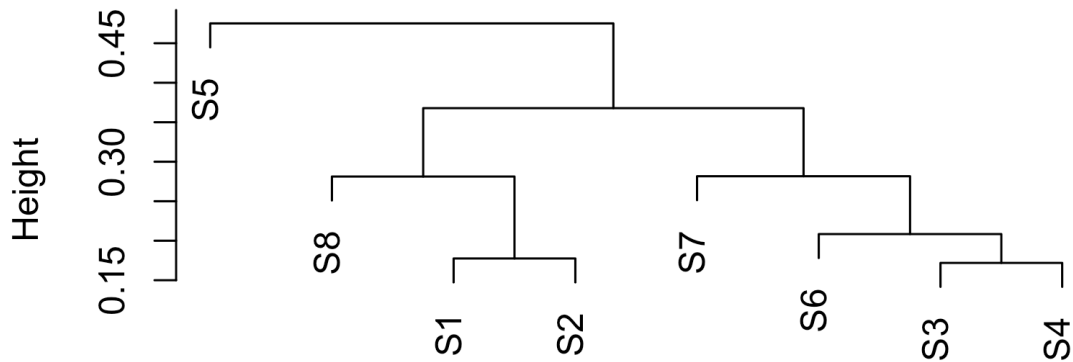
Graph dissimilarities



Same again, but hierarchical clustering

```
hc <- hclust(d_matrix |> select(-from) |> as.dist())  
plot(hc)
```

Cluster Dendrogram



```
as.dist(select(d_matrix, -from))
hclust (*, "complete")
```

Edge coherence

```
tb_edge_coherence <-
  edges_with_metrics |>
  summarise(
    n = sum(n),
    n_desc = sum(n_desc),
    max_n = sum(max_n),
    aligned = mean(swached == "aligned"),
    contested = mean(swached != "aligned")
  ) |>
  mutate(edges_included = n / max_n,
         desc_incllude = n_desc / max_n)
tb_edge_coherence
```

A tibble: 1 x 7

	n	n_desc	max_n	aligned	contested	edges_included	desc_incllude
	<int>	<int>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
1	204	258	258	0.938	0.0622	0.791	1

Generally when two nodes in the model were selected by different participants the same edge was drawn in 79.07% of the time. A handful of potential edges (6.22%) were included, but assigned a different direction of causal flow. In 100% of the cases where a path $A \rightarrow B$ was included in a model, then there existed a path from A to B in every other model that included those two nodes, just sometimes through a mediator(s) variable.

Merging graphs

```
merged_all_graph_layout <- create_layout(merged_all_graph, layout = "sugiyama")
merged_all_graph_layout_adjusted <-
  merged_all_graph_layout |>
  mutate(
    # x = if_else(name == "S.Post.Fb", x + 20, x),
    y = if_else(name == "S.Act.Fb.Post", y + 4, y)
  ) |>
  mutate(
    x = if_else(name == "S.Act", x + 20, x),
    y = if_else(name == "S.Act", y - 4, y)
  )
ptemp <- merged_all_graph_layout_adjusted |>
  ggraph() +
  geom_edge_fan(aes(alpha = edge_w), strength = 0.2,
    arrow = arrow(length = unit(2, 'mm'), type = "closed"),
    start_cap = circle(3, 'mm'),
    end_cap = circle(3, 'mm')) +
  geom_node_point(aes(colour = node_type, size = node_w)) +
  # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.15, 0.8)) +
  scale_alpha_continuous(range = c(0.3, 1)) +
  theme(legend.position = "none")

b <- ggplot_build(ptemp)
x_limits_fine <- b$layout$panel_params[[1]]$x.range
y_limits_fine <- b$layout$panel_params[[1]]$y.range

plot_fine_graph_from_layout <- function(lyt, weight_nodes = TRUE) {
  ppp <-
  lyt |>
```



```

ggraph()

if (weight_nodes) {
  pp <- ppp +
    geom_node_point(aes(colour = node_type,
                        size = node_w)) +
    scale_size(name = "point-size") +
    geom_edge_fan(aes(
      alpha = edge_p_w * edge_w + edge_w,
      edge_linetype = factor(switched)),
      strength = 0.2,
      arrow = arrow(length = unit(1, 'mm'),
                    type = "closed"),
      start_cap = circle(3, 'mm'),
      end_cap = circle(3, 'mm'))
} else {
  pp <- ppp +
    geom_node_point(aes(colour = node_type)) +
    geom_edge_fan(strength = 0.2, alpha = 0.5,
                  arrow = arrow(length = unit(1, 'mm'),
                                type = "closed"),
                  start_cap = circle(1, 'mm'),
                  end_cap = circle(1, 'mm'))
}

pp +
  scale_colour_okabe_ito() +
  # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.15,0.8)) +
  scale_alpha_continuous(range = c(0.3,1)) +
  theme(legend.position = "none",
        plot.margin = margin(t = 0, r = 0, b = 0, l = 0, unit = "pt")) +
  xlim(x_limits_fine) +
  ylim(y_limits_fine)
}

g_plot_fine <- function(g, lyt = merged_all_graph_layout_adjusted, weight_nodes = TRUE) {
  create_layout(g, layout = "sugiyama") |>
    left_join(lyt |> select(x,y,name,node_group,node_type,node_w), by = "name",
              suffix = c(".default", "")) |>
    plot_fine_graph_from_layout(weight_nodes = weight_nodes)
}

```

```

}

pMergedFineLegend <- merged_all_graph_layout_adjusted |>
  g_plot_fine() +
  theme(legend.position = "bottom") +
  guides(size = "none", edge_width = "none", alpha = "none", edge_alpha = "none",
         color = guide_legend(nrow = 1)) +
  theme(legend.title = element_blank())
pMergedFineLegend

```



--> contested ● Assessment data ● Learning data ● Learning design ● Teacher ● Help

```

pMergedFineLegendTall <- merged_all_graph_layout_adjusted |>
  g_plot_fine() +
  theme(legend.position = "right") +
  guides(size = "none", alpha = "none", edge_alpha = "none", edge_width = "none",
         color = guide_legend(ncol = 2)) +
  theme(legend.title = element_blank())
pMergedFine <- pMergedFineLegend +
  theme(legend.position = "none")

pMergedFineLabelled <- pMergedFine +

```


[illegible]

```
Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
family 'Arial Narrow' not found in PostScript font database
Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
family 'Arial Narrow' not found in PostScript font database
Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
family 'Arial Narrow' not found in PostScript font database
Warning in grid.Call(C_textBounds, as.graphicsAnnot(x$label), x$x, x$y, : font
family 'Arial Narrow' not found in PostScript font database
```



```

# Plot border
# theme(
#   plot.background = element_rect(
#     colour = "grey70",
#     fill = NA,
#     linewidth = 1
#   )
# )

hl_text_size <- 1.5
hl_text_alpha <- 0.7

p2peerHighlight <- p2 +
  geom_node_point(size = 3, shape = 1, alpha = 0.6,
    data = p2$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 5, shape = 1, alpha = 0.3,
    data = p2$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 7, shape = 1, alpha = 0.1,
    data = p2$data |> filter(name == "H.Peer")) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    size = hl_text_size, alpha = hl_text_alpha,
    data = p2$data |> filter(name == "H.Peer") |>
      inner_join(name_node_nice_name, by = "name"), repel = T)
p3peerHighlight <- p3 +
  geom_node_point(size = 3, shape = 1, alpha = 0.6,
    data = p3$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 5, shape = 1, alpha = 0.3,
    data = p3$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 7, shape = 1, alpha = 0.1,
    data = p3$data |> filter(name == "H.Peer")) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    size = hl_text_size, alpha = hl_text_alpha,
    data = p3$data |> filter(name == "H.Peer") |>
      inner_join(name_node_nice_name, by = "name"), repel = T)
p4peerHighlight <- p4 +
  geom_node_point(size = 3, shape = 1, alpha = 0.6,
    data = p4$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 5, shape = 1, alpha = 0.3,
    data = p4$data |> filter(name == "H.Peer")) +
  geom_node_point(size = 7, shape = 1, alpha = 0.1,
    data = p4$data |> filter(name == "H.Peer")) +
  geom_node_text(aes(label = nice_name, colour = node_type),

```

```

      size = hl_text_size , alpha = hl_text_alpha,
data = p4$data |> filter(name == "H.Peer") |>
  inner_join(name_node_nice_name, by = "name"), repel = T)

p7KnHighlight <- p7 +
  geom_node_point(size = 3, shape = 5, alpha = 0.6,
data = p7$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 5, shape = 5, alpha = 0.3,
data = p7$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 7, shape = 5, alpha = 0.1,
data = p7$data |> filter(str_detect(name, "Kn"))) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    size = hl_text_size , alpha = hl_text_alpha,
data = p7$data |> filter(str_detect(name, "Kn")) |>
  inner_join(name_node_nice_name, by = "name"), repel = T)

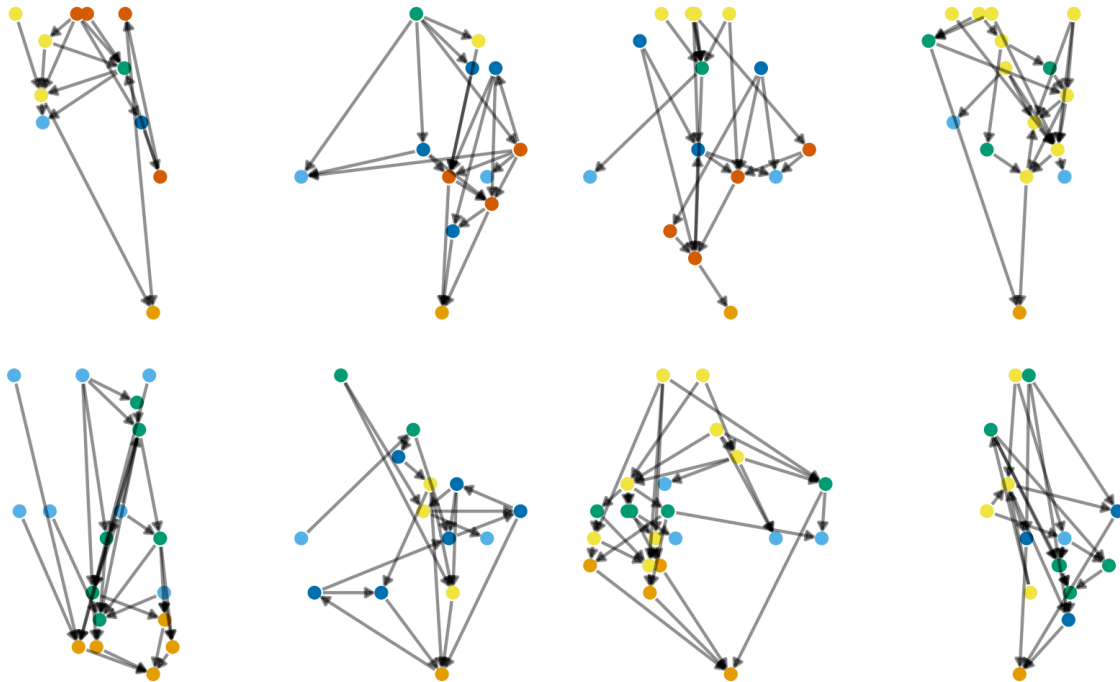
p5KnHighlight <- p5 +
  geom_node_point(size = 3, shape = 5, alpha = 0.6,
data = p5$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 5, shape = 5, alpha = 0.3,
data = p5$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 7, shape = 5, alpha = 0.1,
data = p5$data |> filter(str_detect(name, "Kn"))) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    size = hl_text_size , alpha = hl_text_alpha,
data = p5$data |> filter(str_detect(name, "Kn")) |>
  inner_join(name_node_nice_name, by = "name"), repel = T)

p6KnHighlight <- p6 +
  geom_node_point(size = 3, shape = 5, alpha = 0.6,
data = p6$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 5, shape = 5, alpha = 0.3,
data = p6$data |> filter(str_detect(name, "Kn"))) +
  geom_node_point(size = 7, shape = 5, alpha = 0.1,
data = p6$data |> filter(str_detect(name, "Kn"))) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    size = hl_text_size , alpha = hl_text_alpha,
data = p6$data |> filter(str_detect(name, "Kn")) |>
  inner_join(name_node_nice_name, by = "name"), repel = T)
#

```



```
pAllFine <- (p1 | p2 | p3 | p4) / (p5 | p6 | p7 | p8)
pAllFine
```



```
# ggsave("dags_All_fine.svg", pAllFine, width = 18, height = 9)
```

Figure: Model graphs, all and merged

```
library(grid)
tag_plot <- function(p, tag, x = 0, y = 0.9, fontsize = 8) {
  p + annotation_custom(
    grid::textGrob(tag, x = x, y = y,
      hjust = 0, vjust = 1, gp = gpar(fontsize = fontsize))
  )
}

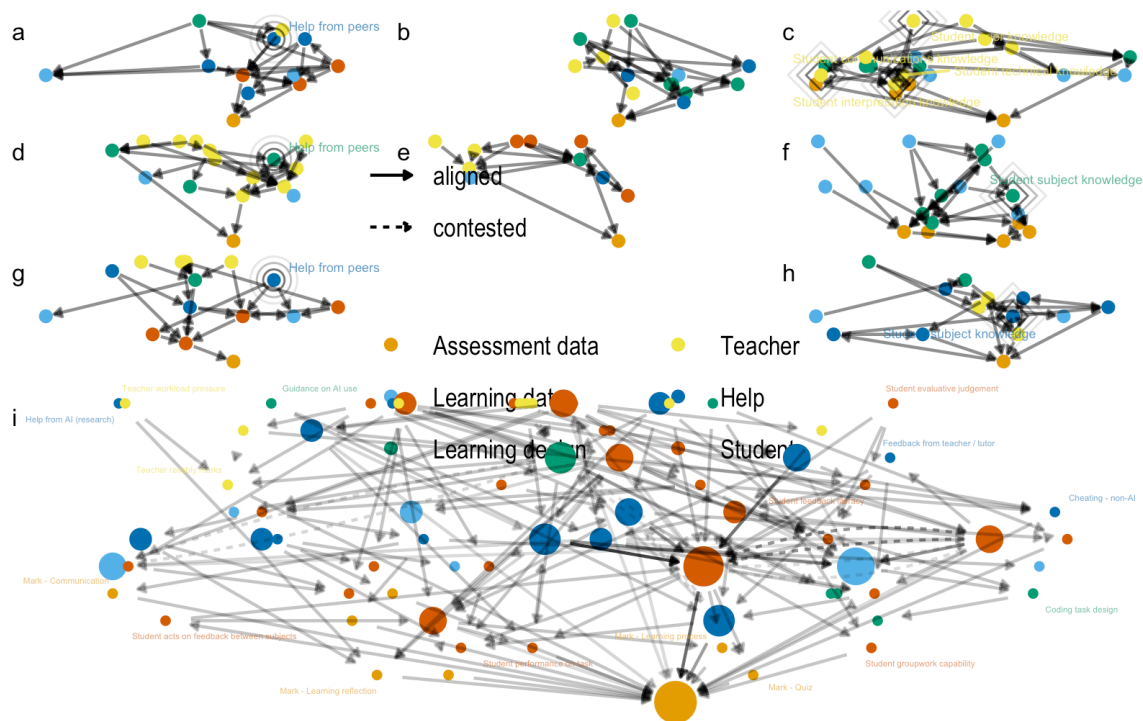
pAll3by3 <-
  (
    tag_plot(p2peerHighlight, "a") |
    tag_plot(p8, "b") |
```

```

        tag_plot(p7KnHighlight, "c")) /
    (
        tag_plot(p4peerHighlight, "d") |
        tag_plot(p1, "e") |
        tag_plot(p5KnHighlight, "f")) /
    (
        tag_plot(p3peerHighlight, "g") |
        leg_tall_plot |
        tag_plot(p6KnHighlight, "h"))

pFineTogetherLabelled <- pAll3by3 / tag_plot(pMergedFineLabelled, "i") + plot_layout(heights = c(1,1,1,3))
pFineTogether <- pAll3by3 / pMergedFine + plot_layout(heights = c(1,1,1,3))
pFineTogetherLabelled

```



```

# ggsave("dags_fine_all_and_merged.png", pFineTogether, width = 9, height = 10, dpi = 900)
# ggsave("dags_fine_all_and_merged.svg", pFineTogether, width = 9, height = 10)
ggsave("figures/FIG--dags_fine_all_and_merged_labelled.jpg", pFineTogetherLabelled, width = 9, height = 10)
# ggsave("dags_fine_all_and_merged_labelled.svg", pFineTogetherLabelled, width = 9, height = 10)

```

```
print("a, d, g are models S2, S3, S4; b, e are S8 and S1; c, f, h are S7, S5, S6. Closest mo
```

```
[1] "a, d, g are models S2, S3, S4; b, e are S8 and S1; c, f, h are S7, S5, S6. Closest mode
```

Common patterns - fine

Most common three-node patterns.

```
merged_all_graph |>
  activate("nodes") |>
  filter(node_w > 3) |>
  activate("edges") |>
  filter(edge_w > 1) |>
  g_plot_fine() +
  geom_node_text(aes(label = nice_name, colour = node_type),
    alpha = hl_text_alpha, repel = TRUE)
```

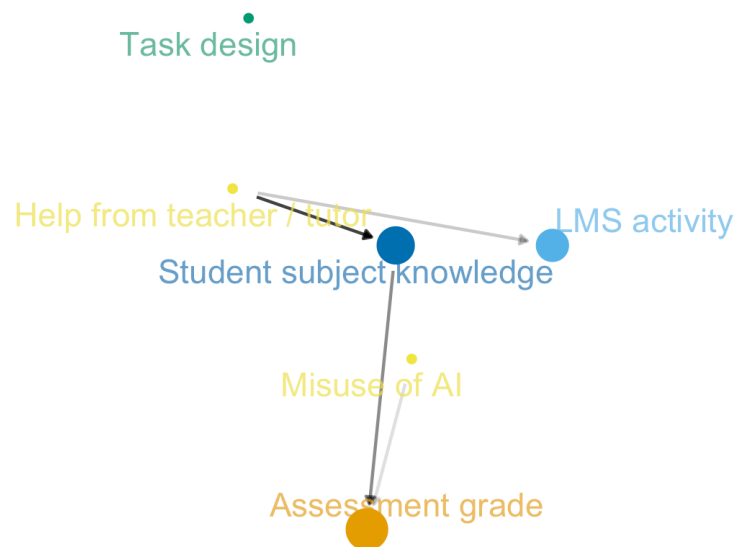


Figure - all graphs, single highlight

```
plot_highlighted_fine_graph <- function(model = 1, LEGEND_POSITION = "none") {  
  MODEL <- paste0("s", model)  
  nodes_in_graph <- all_nodes |>  
    filter(Model == MODEL) |>  
    pull(name)  
  edges_in_graph <- all_edges |>  
    filter(Model == MODEL) |>  
    select(from, to)  
  
  edges_in_graph_indexed <-  
    edges_in_graph |>  
    mutate(  
      from = match(from, merged_all_graph %N>% as_tibble() |> pull(name)),  
      to = match(to, merged_all_graph %N>% as_tibble() |> pull(name))  
    )  
  
  g_highlight <- merged_all_graph %N>%  
    mutate(  
      node_highlight = if_else(  
        name %in% nodes_in_graph, 2, 1  
      )) %E>%  
    left_join(  
      edges_in_graph_indexed |>  
        mutate(edge_highlight = 2)  
    ) |>  
    mutate(edge_highlight = replace_na(edge_highlight, 1)) |>  
    create_layout("sugiyama")  
  
  g_highlight |>  
    ggraph() +  
    geom_edge_fan(aes(  
      # edge_linetype = factor(switche),  
      alpha = edge_highlight),  
      strength = 0.2,  
      arrow = arrow(length = unit(1, 'mm'),  
        type = "closed"),  
      start_cap = circle(3, 'mm'),  
      end_cap = circle(3, 'mm')) +  
    geom_node_label(  
      aes(label = nice_name,
```

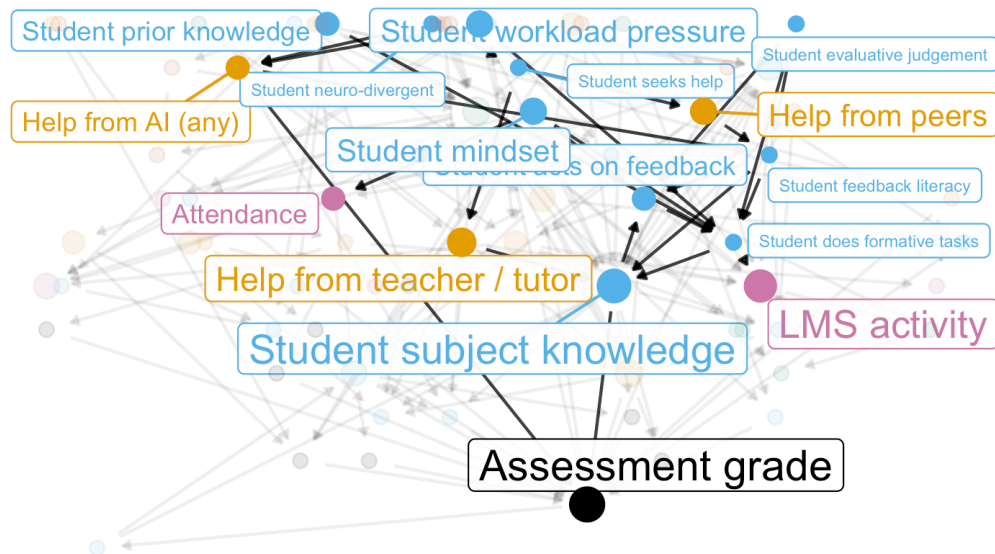
```

        color = node_type,
        size = node_w),
    alpha = 0.95,
    data = g_highlight |>
      filter(node_highlight == 2),
    repel = T, max.overlaps = 30,
    show.legend = FALSE) +
  geom_node_point(aes(colour = node_type,
                      size = node_w,
                      alpha = node_highlight)) +
  scale_size(name = "point-size", range = c(2,5)) +
  scale_color_okabe_ito(order = c(9, 7,1, 2,3, 6, 8,5, 4)) +
  # scale_color_npg() +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.08,0.8)) +
  scale_alpha_continuous(range = c(0.1,1.0)) +
  # theme(legend.position = "none",
  #       plot.margin = margin(t = 0, r = 0, b = 0, l = 0, unit = "pt")) +
  theme(legend.position = LEGEND_POSITION) +
  guides(size = "none", edge_width = "none", alpha = "none", edge_alpha = "none",
         color = guide_legend(ncol = 1)) +
  theme(legend.title = element_blank())
}

plot_highlighted_fine_graph(4)

```

Joining with `by = join\_by(from, to)`



```
# +
#   xlim(x_limits_fine) +
#   ylim(y_limits_fine)

plot_highlighted_coarse_graph <- function(model = 1, LEGEND_POSITION = "none") {
  MODEL <- paste0("s", model)
  node_groups_in_graph <- all_nodes |>
    filter(Model == MODEL) |>
    mutate(name = code_node_group(name)) |>
    distinct(name) |>
    pull(name)
  edges_in_coarse_graph <- all_edges |>
    filter(Model == MODEL) |>
    select(from, to) |>
    mutate(across(from:to, code_node_group))

  edges_in_graph_indexed <-
    edges_in_coarse_graph |>
    mutate(
```

```

    from = match(from, merged_coarse_models %N>% as_tibble() |> pull(name)),
    to = match(to, merged_coarse_models %N>% as_tibble() |> pull(name))
  )

edge_integers <-
  edges_in_graph_indexed |>
  mutate(p2p3 = (2 ** from) * (3 ** to) ) |>
  pull(p2p3)

node_types <- merged_all_graph %N>%
  as_tibble() |>
  distinct(node_group, node_type)

g_highlight <- merged_coarse_models %N>%
  inner_join(node_types |> rename(name = node_group),
    by = "name") |>
  mutate(
    node_highlight = if_else(
      name %in% node_groups_in_graph, 2, 1
    ) %E>%
  )
  mutate(edge_highlight = if_else(
    ((2 ** from) * (3 ** to)) %in% edge_integers, 2, 1
  )) |>
  create_layout("sugiyama")

g_highlight |>
  ggraph() +
  geom_edge_fan(aes(
    # edge_linetype = factor(switced),
    alpha = edge_highlight),
    strength = 0.2,
    arrow = arrow(length = unit(1, 'mm'),
      type = "closed"),
    start_cap = circle(3, 'mm'),
    end_cap = circle(3, 'mm')) +
  geom_node_label(
    aes(label = nice_name,
      color = node_type,
      size = node_w),
    alpha = 0.95,
    data = g_highlight |>
      filter(node_highlight == 2),

```

```

    repel = T, max.overlaps = 30,
    show.legend = FALSE) +
  geom_node_point(aes(colour = node_type,
                      size = node_w,
                      alpha = node_highlight)) +
  scale_size(name = "point-size", range = c(2,5)) +
  # scale_color_npg() +
  scale_color_okabe_ito(order = c(9, 7,1, 2,3, 6, 8,5, 4)) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.08,0.8)) +
  scale_alpha_continuous(range = c(0.1,1.0)) +
  # theme(legend.position = "none",
  #       plot.margin = margin(t = 0, r = 0, b = 0, l = 0, unit = "pt")) +
  theme(legend.position = LEGEND_POSITION) +
  guides(size = "none", edge_width = "none", alpha = "none", edge_alpha = "none",
         label = "none",
         color = guide_legend(ncol = 1)) +
  theme(legend.title = element_blank())
}

# Model used in paper
p_merged_highlighted_fine <- plot_highlighted_fine_graph(5, LEGEND_POSITION = "right")

```

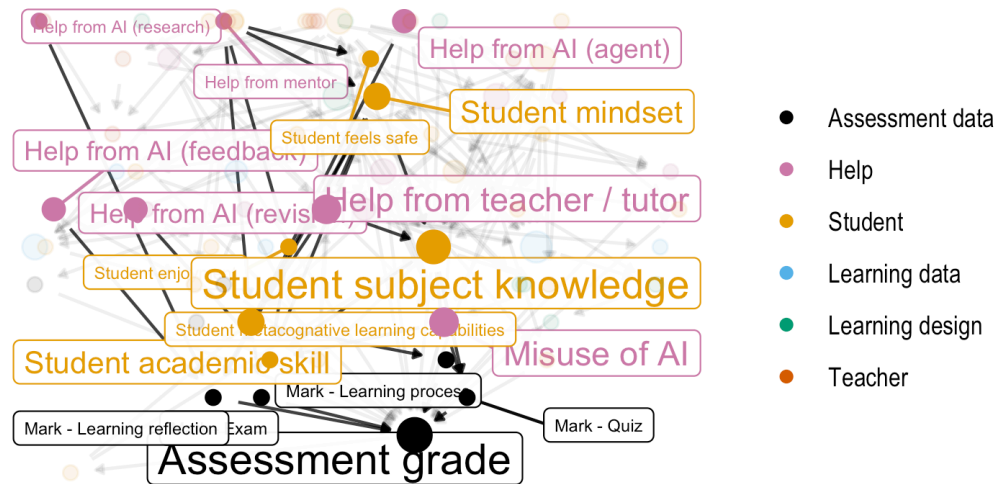
Joining with `by = join\_by(from, to)`

```

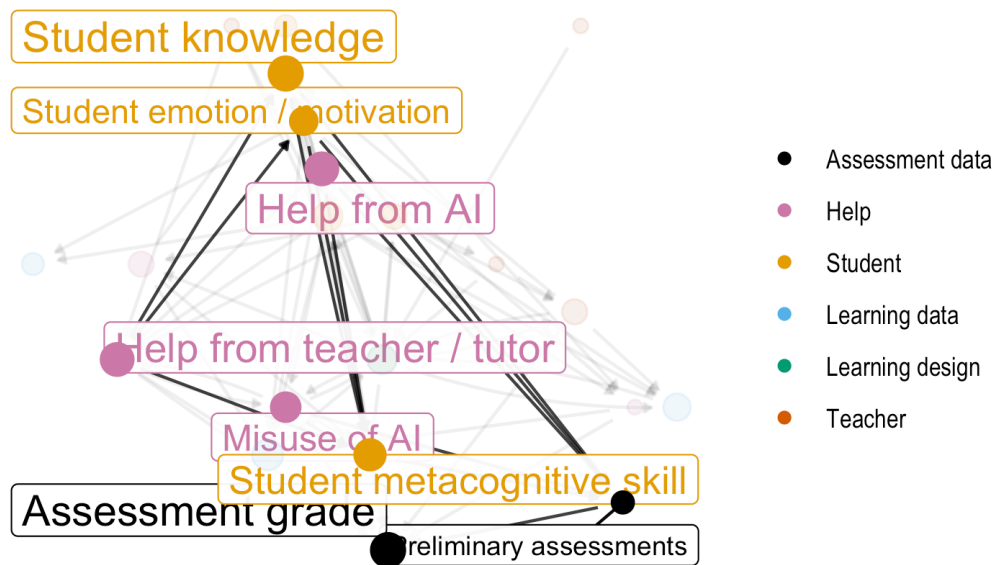
p_merged_highlighted_coarse <- plot_highlighted_coarse_graph(5, LEGEND_POSITION = "right")
p_merged_highlighted_fine + ggtitle("Model used in paper (5)")

```


Model used in paper (5)



p_merged_highlighted_coarse



```
ggsave(plot = p_merged_highlighted_fine,
        filename = "figures/FIG--merged-fine-with-1-highlighted.jpg",
        width = 8, height = 5, dpi = 600)

ggsave(plot = p_merged_highlighted_coarse,
        filename = "figures/FIG--merged-coarse-with-1-highlighted.jpg",
        width = 8, height = 5, dpi = 600)
```

```
# Generating all
p_fine_show_and_save <- function(n){
  p_temp <- plot_highlighted_fine_graph(n, LEGEND_POSITION = "right") + ggtitle(paste0("Model", n))
  ggsave(plot = p_temp,
        filename = paste0("figures/plt--merged-fine-with-", n, "-highlighted.png"),
        width = 8, height = 5, dpi = 600)
  return(p_temp)
}

p_coarse_show_and_save <- function(n){
  p_temp <- plot_highlighted_coarse_graph(n, LEGEND_POSITION = "right") + ggtitle(paste0("Model", n))
  ggsave(plot = p_temp,
        filename = paste0("figures/plt--merged-coarse-with-", n, "-highlighted.png"),
        width = 8, height = 5, dpi = 600)
  return(p_temp)
}
```

```

        width = 8, height = 5, dpi = 600)
    return(p_temp)
}

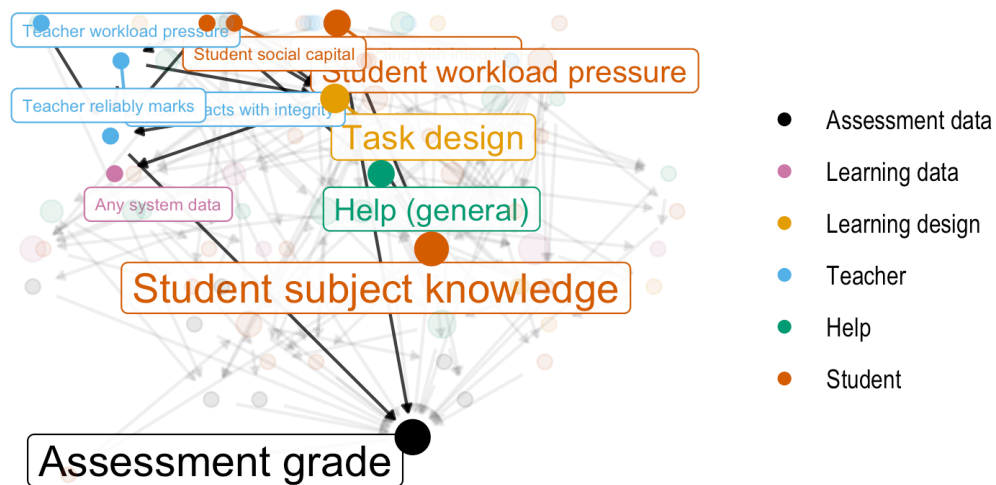
```

Highlighting all models in turn

```
p_fine_show_and_save(1)
```

Joining with `by = join\_by(from, to)`

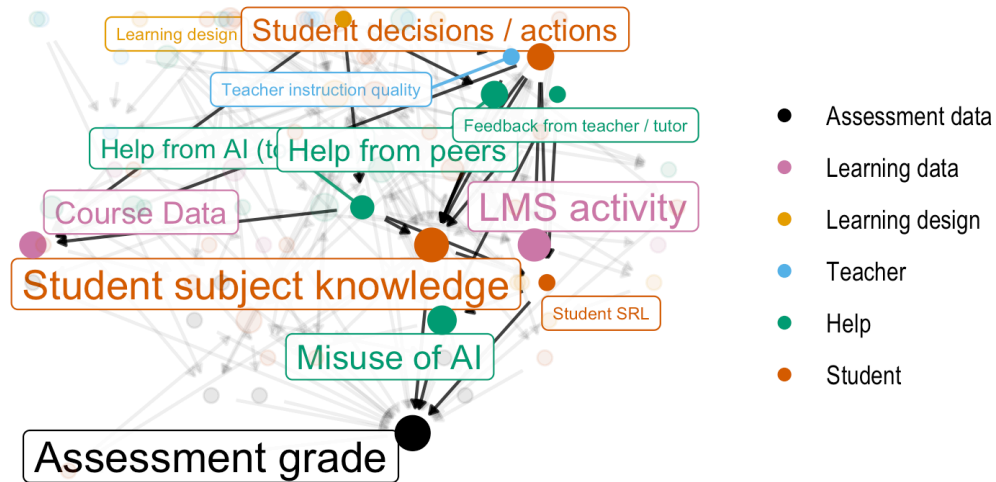
Model 1 highlighted



```
p_fine_show_and_save(2)
```

Joining with `by = join\_by(from, to)`

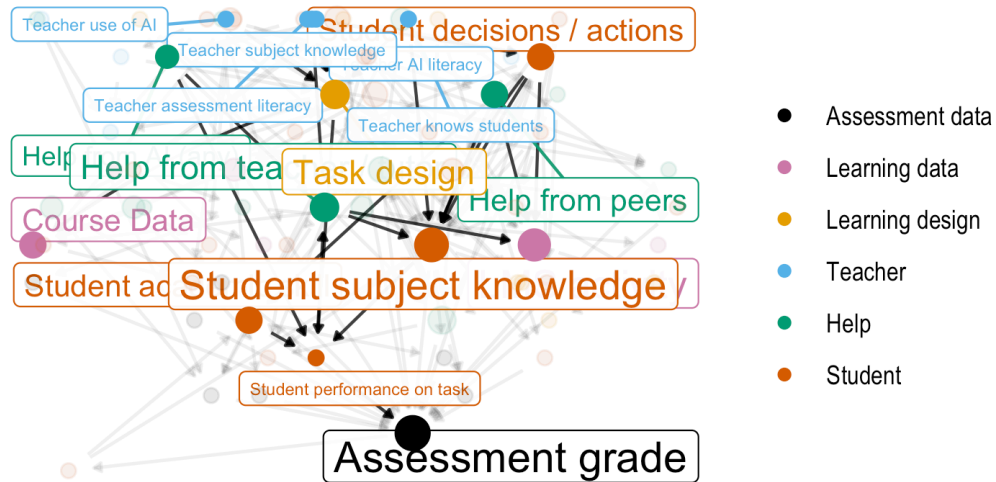
Model 2 highlighted



```
p_fine_show_and_save(3)
```

```
Joining with `by = join_by(from, to)`
```

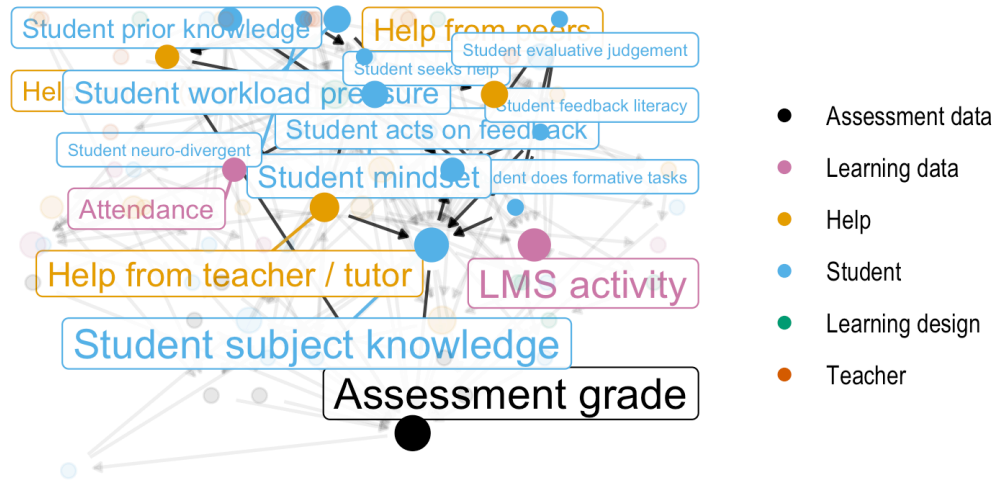
Model 3 highlighted



```
p_fine_show_and_save(4)
```

```
Joining with `by = join_by(from, to)`
```

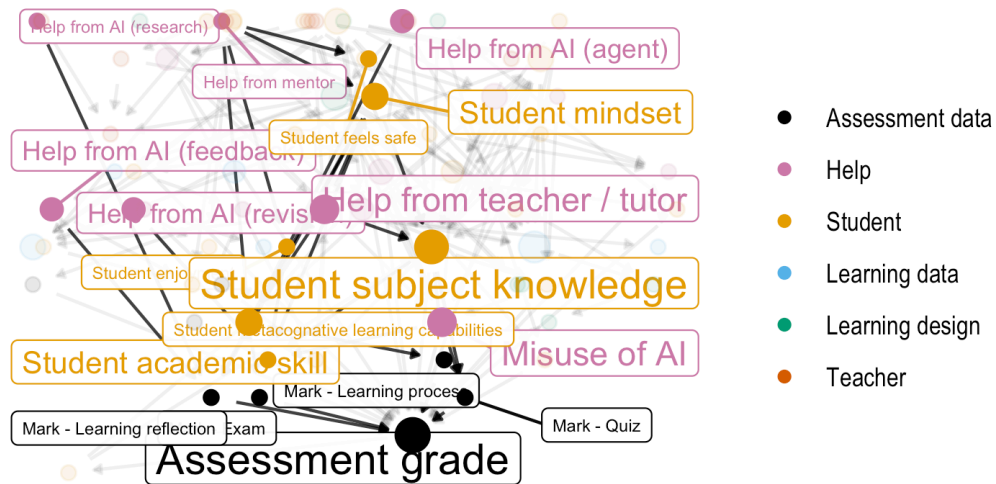
Model 4 highlighted



```
p_fine_show_and_save(5)
```

```
Joining with `by = join_by(from, to)`
```

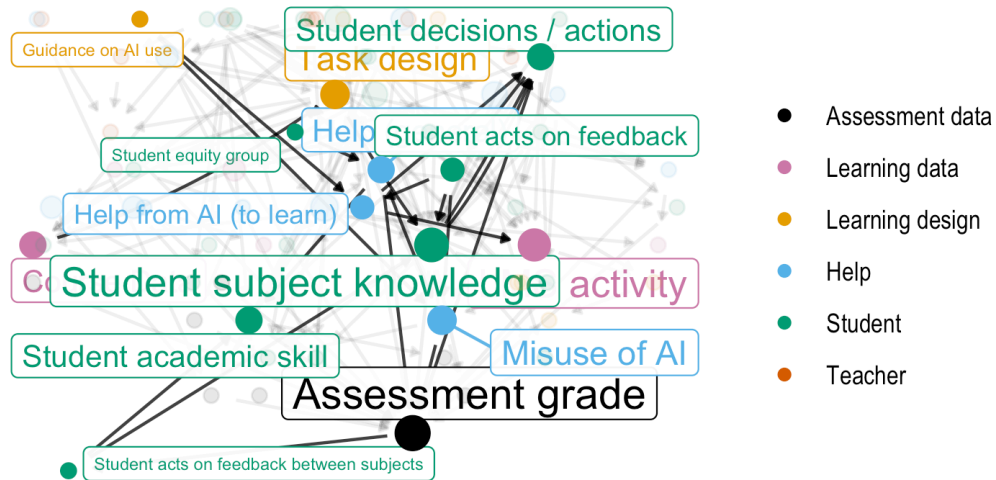
Model 5 highlighted



```
p_fine_show_and_save(6)
```

```
Joining with `by = join_by(from, to)`
```

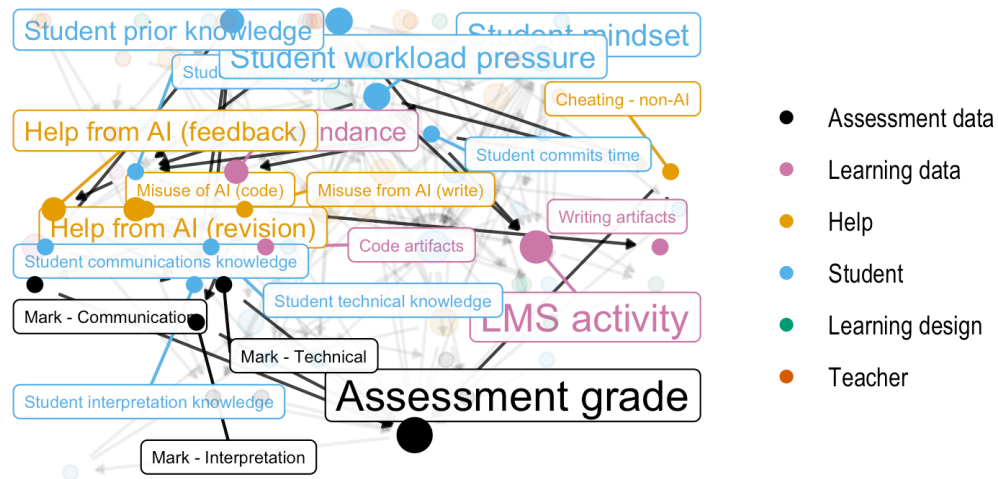
Model 6 highlighted



```
p_fine_show_and_save(7)
```

```
Joining with `by = join_by(from, to)`
```

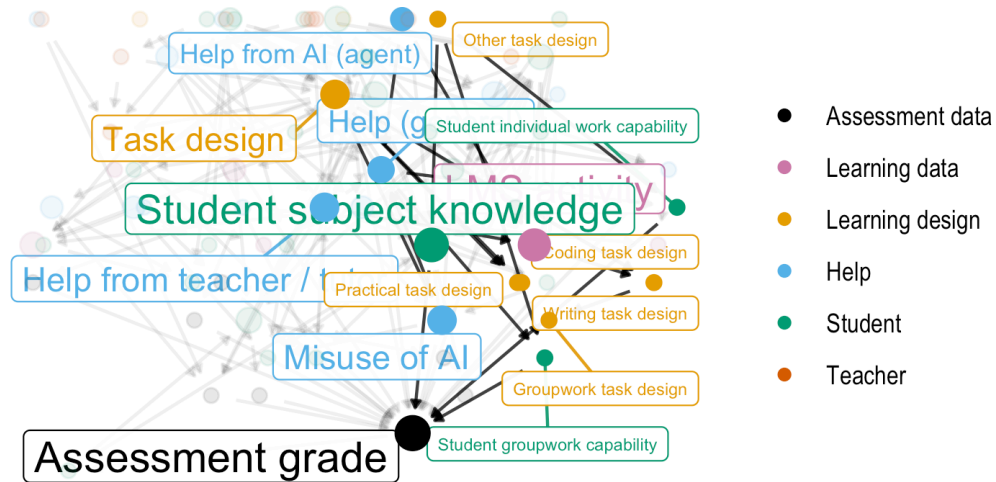

Model 7 highlighted



```
p_fine_show_and_save(8)
```

```
Joining with `by = join_by(from, to)`
```

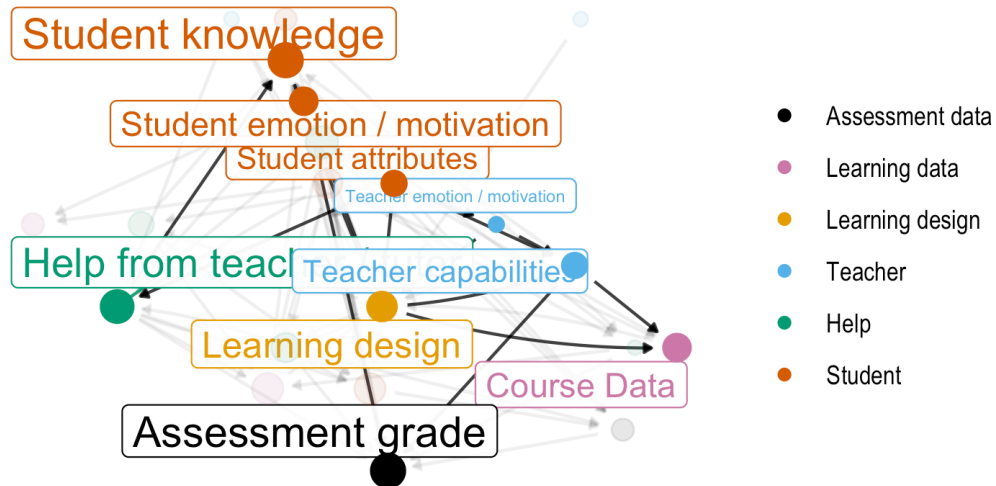
Model 8 highlighted



Highlighting all coarsened models in turn

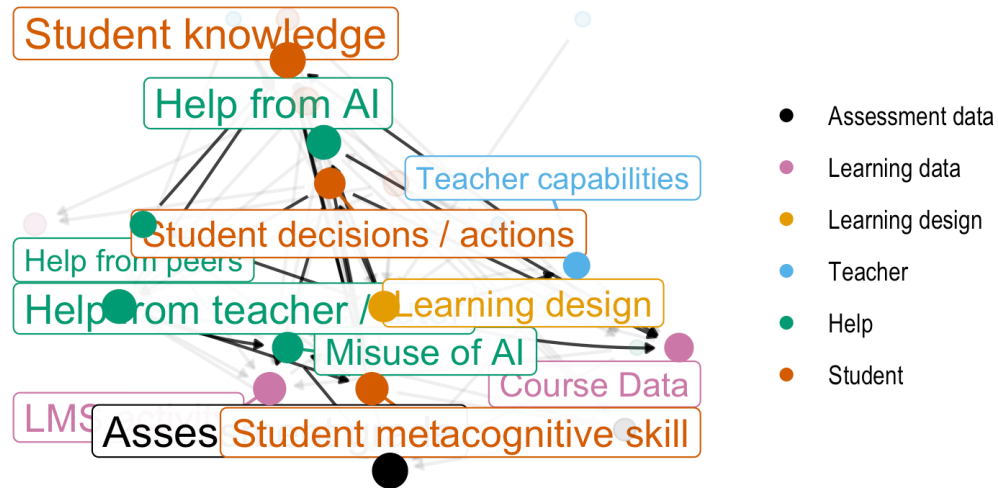
```
p_coarse_show_and_save(1)
```

Coarsened model 1 highlighted



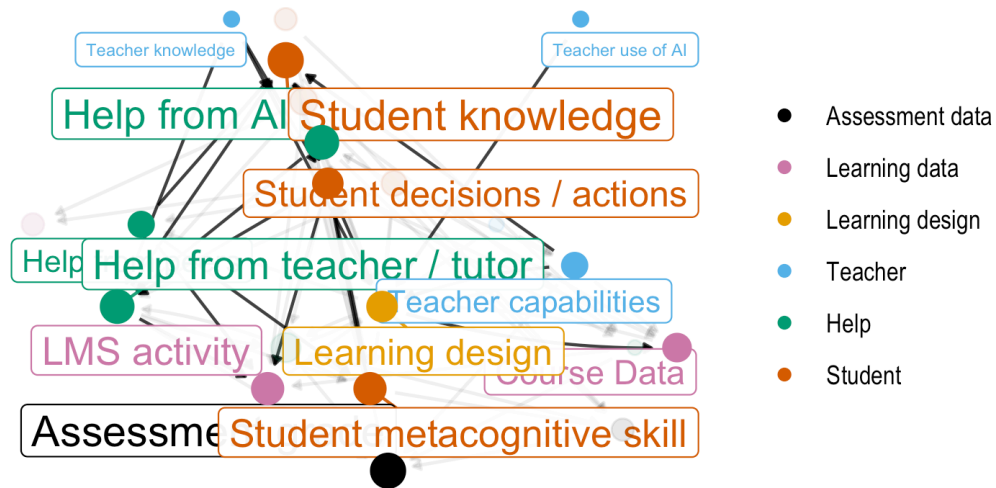
```
p_coarse_show_and_save(2)
```

Coarsened model 2 highlighted



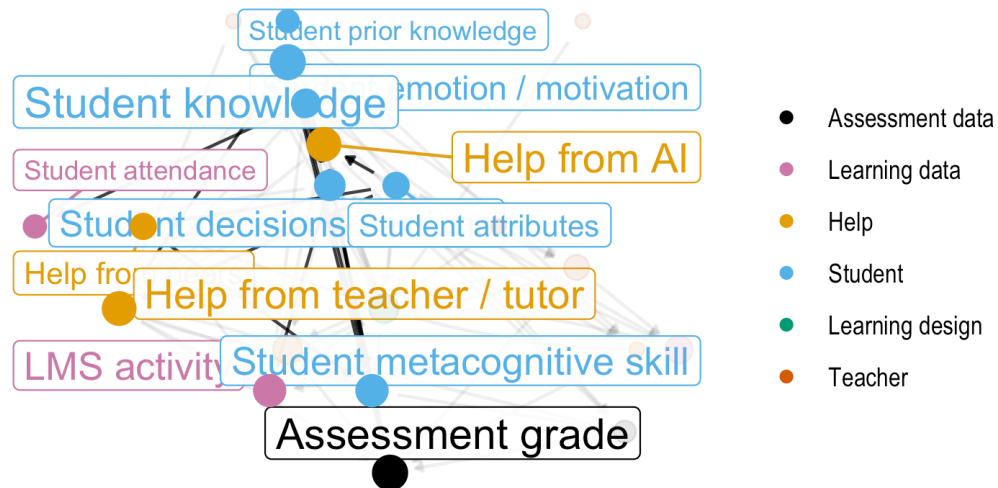
```
p_coarse_show_and_save(3)
```

Coarsened model 3 highlighted



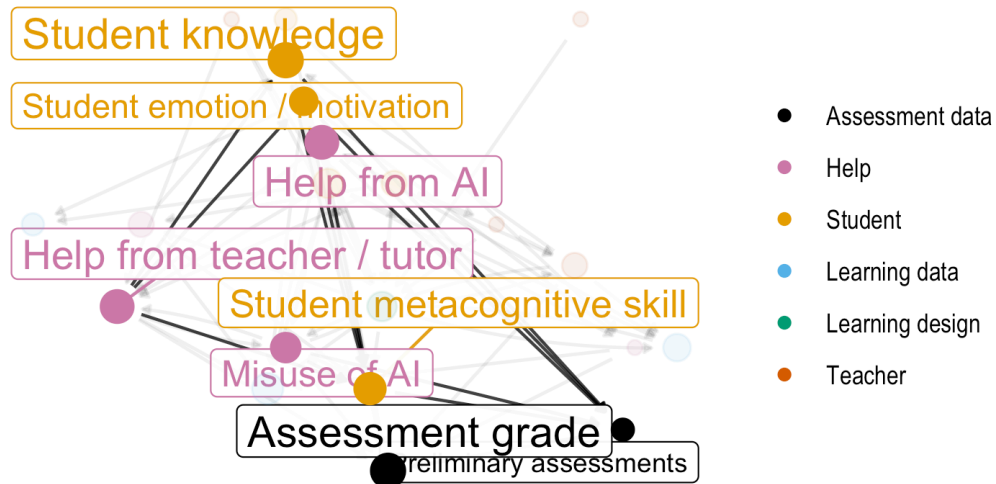
```
p_coarse_show_and_save(4)
```

Coarsened model 4 highlighted



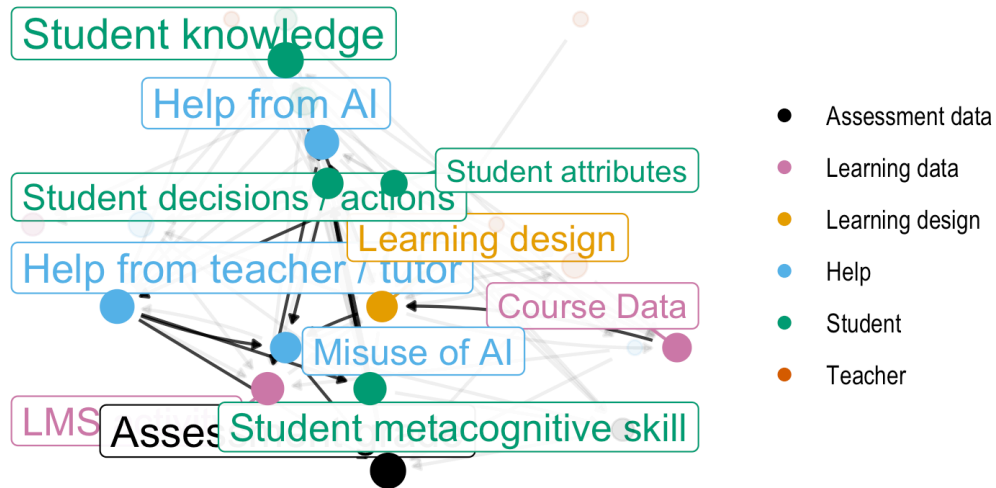
```
p_coarse_show_and_save(5)
```

Coarsened model 5 highlighted



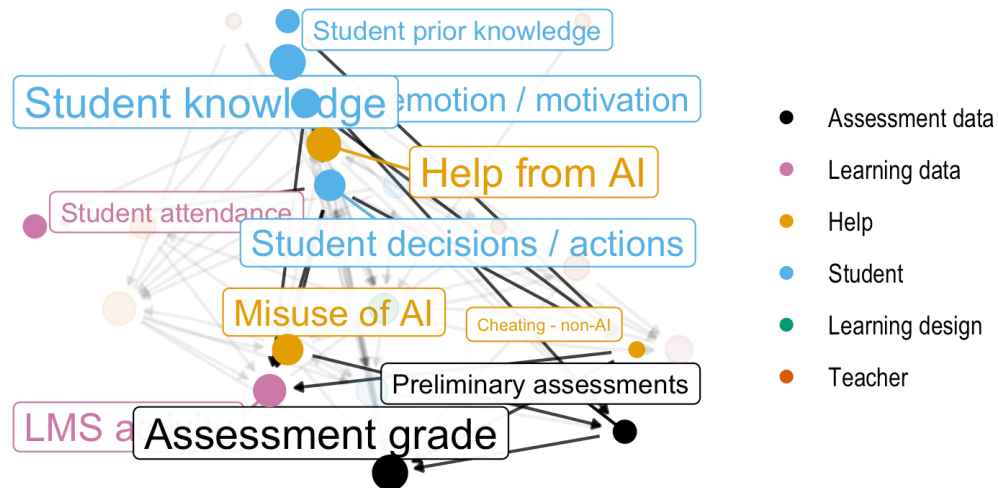
```
p_coarse_show_and_save(6)
```

Coarsened model 6 highlighted



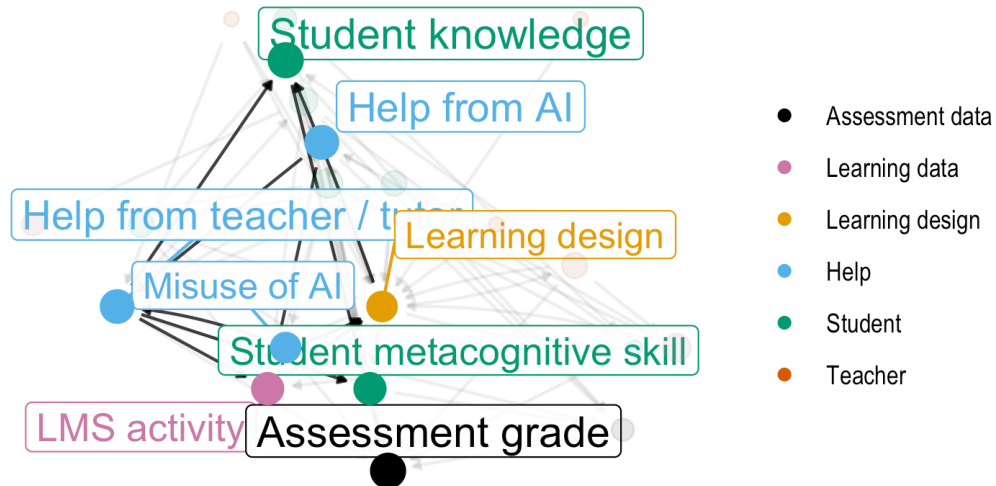
```
p_coarse_show_and_save(7)
```


Coarsened model 7 highlighted



```
p_coarse_show_and_save(8)
```

Coarsened model 8 highlighted



Visualising coarsened graphs

```
merged_coarse_layout <-
  merged_all_graph |>
  g_coarsen_dag_by_ng() |>
  create_layout(layout = "sugiyama") |>
  mutate(
    x = case_when(
      name == "CD.Att" ~ 18.5,
      name == "T.Aff.WrkLd" ~ x - 5,
      name == "H.AI.good" ~ 27,
      TRUE ~ x),
    y = case_when(
      name == "CD.Att" ~ y + 1,
      name == "Grade" ~ y - 2.5,
      name == "CD.Str" ~ 3,
      name == "S.EM" ~ 5,
      TRUE ~ y)
  )
```

```

p_temp <- merged_coarse_layout |>
  ggraph() +
    geom_edge_fan(aes(alpha = edge_w), strength = 0.2,
      arrow = arrow(length = unit(2, 'mm')),
      start_cap = circle(3, 'mm'),
      end_cap = circle(3, 'mm')) +
    geom_node_point(aes(colour = node_type, size = node_w)) +
    scale_color_okabe_ito(order = c(9, 7, 1, 2, 3, 6, 8, 5, 4)) +
    # scale_color_npg(drop = FALSE) +
    # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
    theme_graph() +
    scale_edge_alpha_continuous(range = c(0.15, 0.8)) +
    scale_alpha_continuous(range = c(0.2, 0.6)) +
    theme(legend.position = "none")

bc <- ggplot_build(p_temp)
x_limits_course <- bc$layout$panel_params[[1]]$x.range
y_limits_course <- bc$layout$panel_params[[1]]$y.range

coarse_plot_from_layout <- function(lyt) {
  lyt |>
    ggraph() +
      geom_edge_fan(aes(alpha = edge_w), strength = 0.2,
        arrow = arrow(length = unit(2, 'mm')),
        start_cap = circle(3, 'mm'),
        end_cap = circle(3, 'mm')) +
      geom_node_point(aes(colour = node_type, size = node_w)) +
      scale_color_okabe_ito(order = c(9, 7, 1, 2, 3, 6, 8, 5, 4), drop = FALSE) +
      # ggsci::scale_color_npg(drop = FALSE) +
      # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
      theme_graph() +
      scale_edge_alpha_continuous(range = c(0.15, 0.8)) +
      scale_alpha_continuous(range = c(0.2, 0.6)) +
      theme(legend.position = "none") +
      xlim(x_limits_course) +
      ylim(y_limits_course)
}

g_plot_coarse <- function(g, lyt = merged_coarse_layout) {
  create_layout(g, layout = "sugiyama") |>
    left_join(lyt |> select(x, y, name, node_type, node_w), by = "name",
      suffix = c(".default", "")) |>

```



```

p3course <- g_coarsen_dag_by_ng(tdag_s3) |> g_plot_coarse()
p4course <- g_coarsen_dag_by_ng(tdag_s4) |> g_plot_coarse()
p5course <- g_coarsen_dag_by_ng(tdag_s5) |> g_plot_coarse()
p6course <- g_coarsen_dag_by_ng(tdag_s6) |> g_plot_coarse()
p7course <- g_coarsen_dag_by_ng(tdag_s7) |> g_plot_coarse()
p8course <- g_coarsen_dag_by_ng(tdag_s8) |> g_plot_coarse()

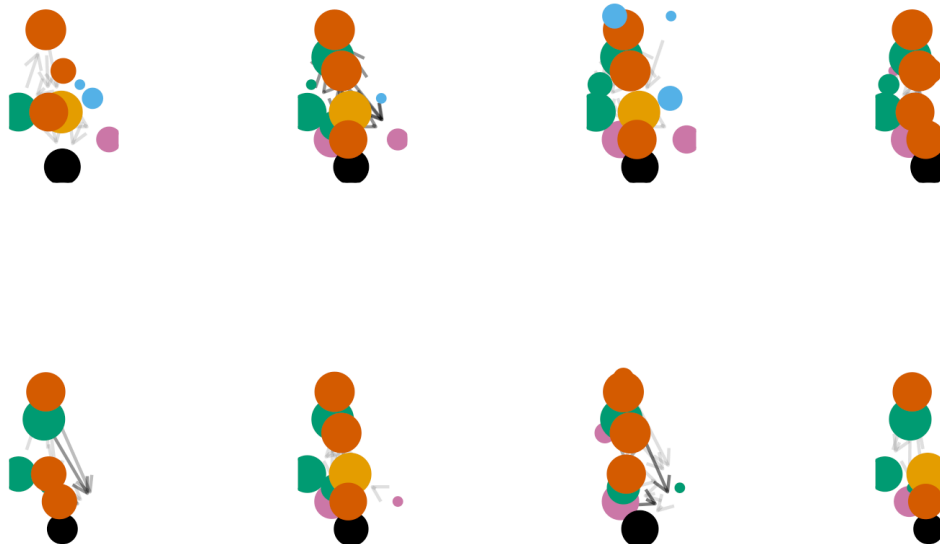
pAllcourse <- (p1course | p2course | p3course | p4course) / (p5course | p6course | p7course
# ggsave("dags_all_course.svg", pAllcourse, width = 18, height = 9)

pAllcourse

```

Warning: Removed 1 row containing missing values or values outside the scale range (`geom\_point()`).

Removed 1 row containing missing values or values outside the scale range (`geom\_point()`).



```

# all node group edges
all_edges |>

```

```
mutate(
  from_g = code_node_group(from),
  to_g = code_node_group(to)) |>
filter(to_g == "LD") |>
filter(from_g == "CD.LMS")
```

```
# A tibble: 3 x 5
  Model from   to           from_g to_g
  <chr> <chr> <chr>         <chr> <chr>
1 s8    CD.LMS LD.Task.Code CD.LMS LD
2 s8    CD.LMS LD.Task.Prac CD.LMS LD
3 s8    CD.LMS LD.Task.Write CD.LMS LD
```

Figure: Consensus DAG

Most common node group patterns - nodes in at least half the models, then looking at the resulting graph of node-groups.

```
node_list_majority_of_models <-
  all_nodes |>
  count(name) |>
  filter(n >= 4) |>
  pull(name)

coarse_filtered_graph_temp <-
  merged_all_graph |>
  activate("nodes") |>
  filter(name %in% node_list_majority_of_models)

coarse_filtered_graph <- tbl_graph(
  nodes = coarse_filtered_graph_temp %N>%
    as_tibble() |>
    mutate(name = code_node_group(name),
           node_type = code_node_type(name)) ,
  edges = coarse_filtered_graph_temp %E>%
    as_tibble() %>%
    mutate(
      from = coarse_filtered_graph_temp %N>% pull(name) %>% .[from],
      to = coarse_filtered_graph_temp %N>% pull(name) %>% .[to]
    ) %>%
  select(from, to, edge_w) |>
```

```

    mutate(across(from:to, code_node_group))) |>
  activate(edges) |>
  mutate(
    switched = map_lgl(row_number(), function(i) {
      from_i <- .E()$from[i]
      to_i    <- .E()$to[i]
      any(from == to_i & to == from_i)
    })
  )

```

```

# Old - not used
lyt_top_node_groups <- create_layout(coarse_filtered_graph, layout = "sugiyama") |>
  mutate(
    x = case_when(
      nice_name == "Student subject knowledge" ~ 0.5,
      nice_name == "Assessment grade" ~ 0.5,
      nice_name == "Help from teacher / tutor" ~ 1,
      nice_name == "Task design" ~ 0.5,
      TRUE ~ x
    ),
    y = case_when(
      nice_name == "Student subject knowledge" ~ 2.5,
      # nice_name == "Assessment grade" ~ 0.5,
      nice_name == "LMS activity" ~ 1,
      nice_name == "Task design" ~ 4,
      TRUE ~ y
    )
  )

p_top_node_groups <- ggraph(lyt_top_node_groups) +
  geom_edge_fan(aes(edge_linetype = factor(switched), alpha = edge_w, ), # alpha = edge_w,
    strength = 0.8,
    arrow = arrow(length = unit(2, 'mm'), type = "closed"),
    start_cap = circle(3, 'mm'),
    end_cap = circle(3, 'mm')) +
  geom_node_point(aes(colour = node_type, size = node_w)) +
  ggsci::scale_color_npg(drop = FALSE) +
  # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.15,0.8)) +
  scale_alpha_continuous(range = c(0.2,0.9)) +
  theme(legend.position = "none") +

```

```

    geom_node_text(aes(label = nice_name, colour = node_type),
                  alpha = 0.9, nudge_y = -.3, repel = T)

# p_top_node_groups
# ggsave("figures/FIG--consensus-DAG.png", p_top_node_groups, dpi = 600, height = 4, width =

```

Weighting by num of models instead This one uses only nodes that are in the majority of models, then coarsens the graph, then aggregates edges based on the number of models that they appear in. The edges must also appear in at least 2 models

```

node_list_majority_of_models <-
  all_nodes |>
  count(name) |>
  filter(n >= 4) |>
  pull(name)

coarse_filtered_graph_ng_filled <-
  tbl_graph( # only nodes that are in most models (4 or more)
    nodes = all_nodes |>
      filter(name %in% node_list_majority_of_models) |>
      mutate( # but then coarsen into node groups
        node_group = code_node_group(name),
        node_type = code_node_type(name)) |>
      count(node_group, node_type) |>
      rename(node_w = n),
    edges = all_edges |> # Only edges between node groups...
      mutate(across(from:to, code_node_group)) |>
      distinct(Model, from, to) |>
      count(from, to) |>
      # filter(n > 1) |> # ...that occur in at least two models
      filter(from %in% code_node_group(node_list_majority_of_models)) |>
      filter(to %in% code_node_group(node_list_majority_of_models)) |>
      rename(edge_w = n)) |>
  activate("edges") |>
  mutate(
    switched = map_lgl(row_number(), function(i) {
      from_i <- .E()$from[i]
      to_i <- .E()$to[i]
      any(from == to_i & to == from_i)
    })
  ) |>

```



```
activate("nodes") |>
inner_join(node_group_nice_name)
```

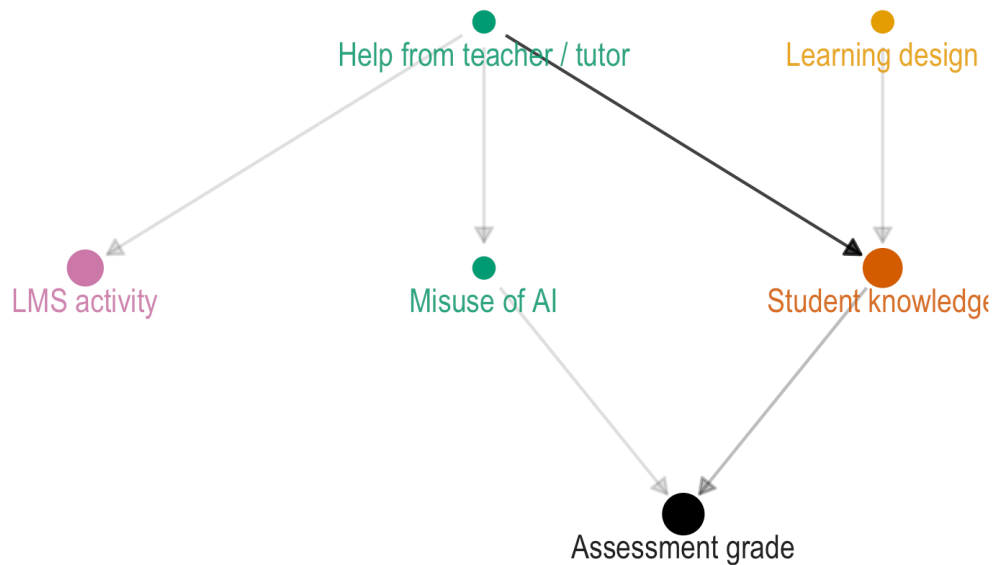
Joining with `by = join\_by(node\_group)`

Filtered - edges must be in at least 2 models

```
lyt_coarse_filtered_ng_filtered <- create_layout(
  coarse_filtered_graph_ng_filled |>
    activate("edges") |>
    filter(edge_w > 1) |>
    mutate(
      switched = map_lgl(row_number(), function(i) {
        from_i <- .E()$from[i]
        to_i <- .E()$to[i]
        any(from == to_i & to == from_i)
      })
    ),
  layout = "sugiyama")

p_top_node_groups_filtered <- ggraph(lyt_coarse_filtered_ng_filtered) +
  geom_edge_fan(aes(edge_linetype = factor(switched), alpha = edge_w, ), # alpha = edge_w,
    strength = 0.8,
    arrow = arrow(length = unit(2, 'mm'), type = "closed"),
    start_cap = circle(3, 'mm'),
    end_cap = circle(3, 'mm')) +
  geom_node_point(aes(colour = node_type, size = node_w)) +
  scale_color_okabe_ito(order = c(9, 7, 1, 2, 3, 6, 8, 5, 4), drop = FALSE) +
  # ggsci::scale_color_npg(drop = FALSE) +
  # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.15, 0.8)) +
  scale_alpha_continuous(range = c(0.2, 0.9)) +
  theme(legend.position = "none") +
  scale_size(range = c(3, 6)) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    alpha = 0.9, nudge_y = -.13, repel = F) +
  xlim(c(min(lyt_coarse_filtered_ng_filtered$x) - 0.15, max(lyt_coarse_filtered_ng_filtered$x) + 0.15))

p_top_node_groups_filtered
```



```

ggsave("figures/FIG--consensus-sub-model-filtered.jpg",
  p_top_node_groups_filtered,
  dpi = 900,
  height = 4, width = 6)

```

All edges between node groups from the majority nodes

```

lyt_coarse_filtered_ng_full <- create_layout(
  coarse_filtered_graph_ng_filled,
  layout = "sugiyama") |>
  select(-x, -y) |>
  inner_join(lyt_coarse_filtered_ng_filtered |> select(x, y, node_group))

```

Joining with `by = join\_by(node\_group)`

```

p_top_node_groups_full <- ggraph(lyt_coarse_filtered_ng_full) +
  geom_edge_fan(aes(edge_linetype = factor(switched), alpha = edge_w, ), # alpha = edge_w,
    strength = 0.8,
    arrow = arrow(length = unit(2, 'mm'), type = "closed"),

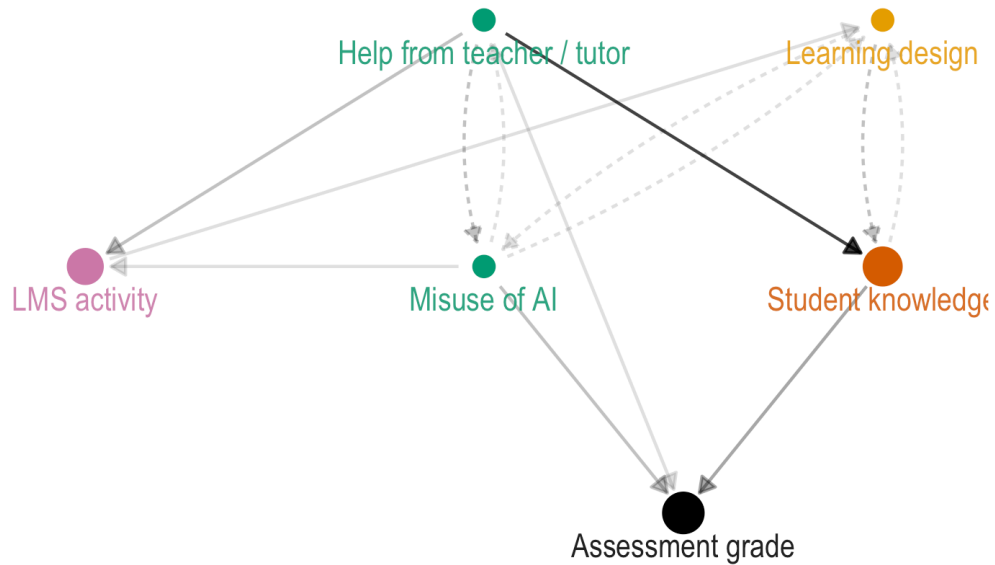
```

```

      start_cap = circle(3, 'mm'),
      end_cap = circle(3, 'mm')) +
geom_node_point(aes(colour = node_type, size = node_w)) +
scale_color_okabe_ito(order = c(9, 7, 1, 2, 3, 6, 8, 5, 4), drop = FALSE) +
  # ggsci::scale_color_npg(drop = FALSE) +
  # geom_node_text(aes(label = name, size = node_w, alpha = node_w), nudge_y = -.5) +
  theme_graph() +
  scale_edge_alpha_continuous(range = c(0.15, 0.8)) +
  scale_alpha_continuous(range = c(0.2, 0.9)) +
  theme(legend.position = "none") +
  scale_size(range = c(3, 6)) +
  geom_node_text(aes(label = nice_name, colour = node_type),
    alpha = 0.9, nudge_y = -.13, repel = F) +
xlim(c(
  min(lyt_coarse_filtered_ng_full$x) - 0.15,
  max(lyt_coarse_filtered_ng_full$x) + 0.15))

```

p_top_node_groups_full



```
ggsave("figures/FIG--consensus-sub-model-full.jpg",
       p_top_node_groups_full,
       dpi = 900,
       height = 4, width = 6)
```

Node groups close to the constructs: - Belonging - Mindset - Collaboration

```
# all nodes
all_nodes_with_metrics |>
  inner_join(name_node_nice_name) |>
  inner_join(node_group_nice_name |> rename(g_nice_name = nice_name)) |>
  select(name, nice_name, g_nice_name, node_group, Model, pagerank_no_outcome, pagerank, br
  arrange(name, Model) |>
  knitr::kable()
```

Joining with `by = join\_by(name)`

Joining with `by = join\_by(node\_group)`

name	nice_name	g_nice_name	node_group	Model	pagerank_no_outcome	pagerank	rel_rank
CD	Any system data	Course Data	CD.Str	S1	0.2598017	0.1254606	5000000
CD.Att	Attendance	Student attendance	CD.Att	S4	0.0229468	0.0435205	3333333
CD.Att	Attendance	Student attendance	CD.Att	S7	0.0348776	0.0360266	5000000
CD.LMS	LMS activity	LMS activity	CD.LMS	S2	0.0489246	0.0601186	7272727
CD.LMS	LMS activity	LMS activity	CD.LMS	S3	0.1242479	0.0777309	1000000
CD.LMS	LMS activity	LMS activity	CD.LMS	S4	0.1161226	0.0715005	3333333
CD.LMS	LMS activity	LMS activity	CD.LMS	S6	0.0633664	0.0518707	8461538
CD.LMS	LMS activity	LMS activity	CD.LMS	S7	0.0403759	0.0398607	6500000
CD.LMS	LMS activity	LMS activity	CD.LMS	S8	0.0576999	0.0576507	1428571
CD.LMS	Code artifacts	LMS activity	CD.LMS	S7	0.0415103	0.0399107	6500000
CD.LMS	Writing artifacts	LMS activity	CD.LMS	S7	0.0752913	0.0477802	6500000
CD.Str	Course Data	Course Data	CD.Str	S2	0.0693898	0.0754602	7272727
CD.Str	Course Data	Course Data	CD.Str	S3	0.0569482	0.0657504	1000000
CD.Str	Course Data	Course Data	CD.Str	S6	0.0276203	0.0375601	8461538
Grade	Assessment grade	Assessment grade	Grade.Outcome	S1	0.0000000	0.1323801	5000000
Grade	Assessment grade	Assessment grade	Grade.Outcome	S2	0.0000000	0.1642707	7272727

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$3	0.0000000	0.07328851	0.0000000
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$4	0.0000000	0.07870015	0.3333333
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$5	0.0000000	0.17302905	0.294118
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$6	0.0000000	0.14392937	0.69231
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$7	0.0000000	0.12754766	0.5000000
Grade	Assessment grade	Assessment grade	Grade.Outcome	\$8	0.0000000	0.15315827	0.142857
Grade.Comm	Mark - Communication	Preliminary assessments	Assessment	\$7	0.0562769	0.04779045	0.5000000
Grade.Exam	Mark - Exam	Preliminary assessments	Assessment	\$5	0.1621565	0.08930042	0.941176
Grade.Interp	Mark - Interpretation	Preliminary assessments	Assessment	\$7	0.1158678	0.07081855	0.5000000
Grade.Proc	Mark - Learning process	Preliminary assessments	Assessment	\$5	0.0762812	0.05738035	0.29412
Grade.Quiz	Mark - Quiz	Preliminary assessments	Assessment	\$5	0.0520260	0.04535071	0.117647
Grade.Ref	Mark - Learning reflection	Preliminary assessments	Assessment	\$5	0.1193653	0.08884072	0.352941
Grade.Tech	Mark - Technical	Preliminary assessments	Assessment	\$7	0.0681343	0.05314044	0.5000000
H	Help (general)	Help from teacher / tutor	H.Tut	S1	0.0534232	0.06432063	0.0000000
H	Help (general)	Help from teacher / tutor	H.Tut	S6	0.0695564	0.06339043	0.846154
H	Help (general)	Help from teacher / tutor	H.Tut	S8	0.0709553	0.06726733	0.0000000
H.AI	Help from AI (any)	Help from AI	H.AI.goo	\$3	0.0202678	0.03802021	0.0000000
H.AI	Help from AI (any)	Help from AI	H.AI.goo	\$4	0.0330791	0.05595002	0.0000000
H.AI.Agen	Help from AI (agent)	Help from AI	H.AI.goo	\$5	0.0293150	0.03258015	0.294118
H.AI.Agen	Help from AI (agent)	Help from AI	H.AI.goo	\$8	0.0311891	0.03843847	0.142857
H.AI.Fb	Help from AI (feedback)	Help from AI	H.AI.goo	\$5	0.0293150	0.03258015	0.294118

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
H.AI.Fb	Help from AI (feedback)	Help from AI	H.AI.good	S7	0.0367908	0.0366805	1500000
H.AI.Res	Help from AI (research)	Help from AI	H.AI.good	S5	0.0293150	0.0325801	15294118
H.AI.Study	Help from AI (revision)	Help from AI	H.AI.good	S5	0.0293150	0.0325801	15294118
H.AI.Study	Help from AI (revision)	Help from AI	H.AI.good	S7	0.0367908	0.0366805	14000000
H.AI.evil	Misuse of AI	Misuse of AI	H.AI.evil	S2	0.2333173	0.0929205	15363636
H.AI.evil	Misuse of AI	Misuse of AI	H.AI.evil	S5	0.0293150	0.0325801	15294118
H.AI.evil	Misuse of AI	Misuse of AI	H.AI.evil	S6	0.1136701	0.0678082	15923077
H.AI.evil	Misuse of AI	Misuse of AI	H.AI.evil	S8	0.0311891	0.0384384	17142857
H.AI.evil.c	Misuse of AI (code)	Misuse of AI	H.AI.evil	S7	0.0367908	0.0366805	14000000
H.AI.evil.w	Misuse from AI (write)	Misuse of AI	H.AI.evil	S7	0.0367908	0.0366805	1500000
H.AI.good	Help from AI (to learn)	Help from AI	H.AI.good	S2	0.0477452	0.0595700	152727273
H.AI.good	Help from AI (to learn)	Help from AI	H.AI.good	S6	0.0841086	0.0572404	153846154
H.Ment	Help from mentor	Help from teacher / tutor	H.Tut	S5	0.0293150	0.0325801	15294118
H.Peer	Help from peers	Help from peers	H.Peer	S2	0.0477452	0.0595700	151545455
H.Peer	Help from peers	Help from peers	H.Peer	S3	0.0202678	0.0380202	14000000
H.Peer	Help from peers	Help from peers	H.Peer	S4	0.0287095	0.0489605	153333333
H.Tut	Help from teacher / tutor	Help from teacher / tutor	H.Tut	S3	0.2051369	0.0891309	151333333
H.Tut	Help from teacher / tutor	Help from teacher / tutor	H.Tut	S4	0.0287095	0.0489605	154000000
H.Tut	Help from teacher / tutor	Help from teacher / tutor	H.Tut	S5	0.0293150	0.0325801	15294118
H.Tut	Help from teacher / tutor	Help from teacher / tutor	H.Tut	S8	0.0311891	0.0384384	17142857

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
H.Tut.Fb	Feedback from teacher / tutor	Help from teacher / tutor	H.Tut	S2	0.0489246	0.0601186	3636364
H.other.ev	Cheating - non-AI	Cheating - non-AI	H.Cheat	S7	0.0458741	0.0437103	2000000
LD	Learning design	Learning design	LD	S2	0.0408079	0.0541609	7272727
LD.AIguid	Guidance on AI use	Learning design	LD	S6	0.0276203	0.0375667	18461538
LD.Task	Task design	Learning design	LD	S1	0.1516042	0.1313263	30000000
LD.Task	Task design	Learning design	LD	S3	0.0863070	0.1109008	6666667
LD.Task	Task design	Learning design	LD	S6	0.0510975	0.0563506	5384615
LD.Task	Task design	Learning design	LD	S8	0.0444445	0.0480480	2857143
LD.Task.Cod	Coding task design	Learning design	LD	S8	0.0601301	0.0560561	3571429
LD.Task.Gr	Groupwork task design	Learning design	LD	S8	0.1845208	0.1225205	714286
LD.Task.Oth	Other task design	Learning design	LD	S8	0.0311891	0.0384384	7142857
LD.Task.Pra	Practical task design	Learning design	LD	S8	0.0601301	0.0560561	3571429
LD.Task.Wri	Writing task design	Learning design	LD	S8	0.0601301	0.0560561	3571429
S.Act	Student decisions / actions	Student decisions / actions	S.Act	S2	0.0477452	0.0595700	1818182
S.Act	Student decisions / actions	Student decisions / actions	S.Act	S3	0.0260103	0.0443605	3333333
S.Act	Student decisions / actions	Student decisions / actions	S.Act	S6	0.1533177	0.0965383	30000000
S.Act.AIstr	Student AI strategy	Student decisions / actions	S.Act	S7	0.0513724	0.0475500	0000000

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
S.Act.Fb	Student acts on feedback	Student decisions / actions	S.Act	S4	0.1870171	0.0908106	0.2666667
S.Act.Fb	Student acts on feedback	Student decisions / actions	S.Act	S6	0.1579403	0.0617006	0.3076923
S.Act.Fb.Fst	Student acts on feedback between subjects	Student decisions / actions	S.Act	S6	0.0276203	0.1095307	0.2307692
S.Act.Frm	Student does formative tasks	Student decisions / actions	S.Act	S4	0.2311575	0.1367896	0.0000000
S.Act.Perf	Student performance on task	Student decisions / actions	S.Act	S3	0.2006032	0.1410574	0.0000000
S.Act.Tim	Student commits time	Student decisions / actions	S.Act	S7	0.0423690	0.0422693	0.2500000
S.Aff.Integ	Student acting with integrity	Student emotion / motivation	S.EM	S1	0.0440604	0.0571705	0.5000000
S.Aff.Joy	Student enjoyment	Student emotion / motivation	S.EM	S5	0.0417738	0.0407302	0.4705882
S.Aff.MindSat	Student mindset	Student emotion / motivation	S.EM	S4	0.0178806	0.0373000	0.5333333
S.Aff.MindSat	Student mindset	Student emotion / motivation	S.EM	S5	0.0719866	0.0590588	0.0000000
S.Aff.MindSat	Student mindset	Student emotion / motivation	S.EM	S7	0.0258742	0.0307403	0.3500000
S.Aff.Safe	Student feels safe	Student emotion / motivation	S.EM	S5	0.0355444	0.0366575	0.294118
S.Aff.WrkIs	Student workload pressure	Student emotion / motivation	S.EM	S1	0.0820852	0.0795087	0.3000000

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
S.Aff.WrkLst	Student workload pressure	Student emotion / motivation	S.EM	S4	0.0178806	0.0373000	0.5333333
S.Aff.WrkLst	Student workload pressure	Student emotion / motivation	S.EM	S7	0.0258742	0.0307403	0.5000000
S.Att.Eq	Student equity group	Student attributes	S.Att	S6	0.0493367	0.0516507	0.153846
S.Att.HSk	Student seeks help	Student attributes	S.Att	S4	0.0254798	0.0466300	0.666667
S.Att.Nro	Student neuro-divergent	Student attributes	S.Att	S4	0.0178806	0.0373000	0.5333333
S.Att.SocCap	Student social capital	Student attributes	S.Att	S1	0.0440604	0.0571705	0.5000000
S.Cp.Aca	Student academic skill	Student metacognitive skill	S.Cp	S3	0.0260103	0.0443605	0.266667
S.Cp.Aca	Student academic skill	Student metacognitive skill	S.Cp	S5	0.0570710	0.0481105	0.1176471
S.Cp.Aca	Student academic skill	Student metacognitive skill	S.Cp	S6	0.0689204	0.0755107	0.7692308
S.Cp.EvJdg	Student evaluative judgement	Student metacognitive skill	S.Cp	S4	0.0178806	0.0373000	0.5333333
S.Cp.Fb	Student feedback literacy	Student metacognitive skill	S.Cp	S4	0.0754671	0.0819901	0.666667
S.Cp.Grp	Student groupwork capability	Student metacognitive skill	S.Cp	S8	0.2169727	0.1173103	0.2142857
S.Cp.Ind	Student individual work capability	Student metacognitive skill	S.Cp	S8	0.0601301	0.0560501	0.5714286
S.Cp.Lrn	Student metacognitive learning capabilities	Student metacognitive skill	S.Cp	S5	0.1277487	0.0893902	0.2588235
S.Cp.SRL	Student SRL	Student metacognitive skill	S.Cp	S2	0.2101735	0.1149702	0.0909091

name	nice_name	g_nice_name	node_group	Model	pagerank_norm	pagerank_rel	rel_rank
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S1	0.0894701	0.0893356	20000000
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S2	0.1574813	0.1396450	60000000
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S3	0.1328617	0.0872409	20000000
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S4	0.1619080	0.1096473	33333333
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S5	0.0508416	0.0440395	1764706
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S6	0.1058251	0.0893202	21538462
S.Kn	Student subject knowledge	Student knowledge	S.Kn	S8	0.0601301	0.0560505	15714286
S.Kn.Com	Student communications knowledge	Student knowledge	S.Kn	S7	0.0470086	0.0437555	50000000
S.Kn.Inter	Student interpretation knowledge	Student knowledge	S.Kn	S7	0.0936112	0.0679259	25000000
S.Kn.Prior	Student prior knowledge	Student prior knowledge	S.Kn.Prior	S4	0.0178806	0.0373000	4053333333
S.Kn.Prior	Student prior knowledge	Student prior knowledge	S.Kn.Prior	S7	0.0258742	0.0307403	35000000
S.Kn.Tech	Student technical knowledge	Student knowledge	S.Kn	S7	0.0626447	0.0529267	35000000
T.Act.Use AI	Teacher use of AI	Teacher use of AI	T.Act	S3	0.0202678	0.0380202	40000000
T.Aff.Integ	Teacher acts with integrity	Teacher emotion / motivation	T.EM	S1	0.0534232	0.0643206	50000000
T.Aff.Wrk	Teacher workload pressure	Teacher emotion / motivation	T.EM	S1	0.0440604	0.0571705	45000000
T.Cp.Inst	Teacher instruction quality	Teacher capabilities	T.Cp	S2	0.0477452	0.0595700	1545455
T.Cp.KnSt	Teacher knows students	Teacher capabilities	T.Cp	S3	0.0202678	0.0380202	40000000
T.Cp.Mark	Teacher reliably marks	Teacher capabilities	T.Cp	S1	0.1780112	0.1418202	21000000

name	nice_name	g_nice_name	node_group	Model	pagerank_no_outcome	pagerank	btw_rel	rel_rank
T.Kn.AI	Teacher AI literacy	Teacher knowledge	T.Kn	S3	0.0202678	0.0380202	0.21	0.000000
T.Kn.Ass	Teacher assessment literacy	Teacher knowledge	T.Kn	S3	0.0202678	0.0380202	0.21	0.000000
T.Kn.Sub	Teacher subject knowledge	Teacher knowledge	T.Kn	S3	0.0202678	0.0380202	0.21	0.000000

Node importance and measurability

```
all_node_groups_aggregated_metrics <-
  all_nodes_with_metrics |>
  group_by(Model, node_group, node_type) |>
  summarise(
    p_traced = mean(measurability == "DW"),
    measurability = (3 * mean(measurability == "DW") +
                     2 * mean(measurability == "0") +
                     mean(measurability == "PO")) / 3,
    pagerank = sum(pagerank),
    pagerank_no_outcome = sum(pagerank_no_outcome),
    btw_rel = sum(btw_rel)) |>
  group_by(node_group, node_type) |>
  mutate(
    n = n(),
    overall_pagerank = sum(pagerank / 8),
    overall_pagerank_no_outcome = sum(pagerank_no_outcome / 8),
    overall_btw = sum(btw_rel)) |>
  ungroup()
```

`summarise()` has regrouped the output.
 i Summaries were computed grouped by Model, node_group, and node_type.
 i Output is grouped by Model and node_group.
 i Use `summarise(.groups = "drop\_last")` to silence this message.
 i Use `summarise(.by = c(Model, node\_group, node\_type))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

Figure: Node importance and data availability

```

d_ng_measurability_importance <-
  all_node_groups_aggregated_metrics |>
  filter(node_group != "Grade.Outcome") |>
  inner_join(node_group_nice_name, by = "node_group") |>
  mutate(node_group = nice_name) |>
  group_by(node_group) |>
  mutate(m_group = mean(measurability)) |>
  ungroup() |>
  mutate(node_group = fct_reorder(node_group, pagerank_no_outcome)) |>
  arrange(desc(node_group)) |>
  filter(n > 3) # need at least half the models with node group

p_top_node_pagerank_no_outcome_measurability <-
  d_ng_measurability_importance |>
  # arrange(Model, node_group) |>
  ggplot(aes(x = pagerank_no_outcome, y = node_group)) +
  geom_boxplot(aes(fill = m_group), colour = "lightgray", size = 0.4, alpha = 0.7) +
  # geom_line(aes(x = pagerank_no_outcome, y = node_group, group = Model), alpha = 0.6) +
  geom_point(aes(colour = measurability), alpha = 0.9) +
  scale_color_gradient(
    low = "darkslateblue",
    # mid = "darkturquoise",
    high = "aquamarine",
    # midpoint = mean(range(d_ng_measurability_importance$measurability)),
    limits = c(0,1),
    breaks = c(0, 1/3, 2/3, 1),
    labels = c("Latent", "Potentially\nobservable", "Observable", "Data\nWarehouse"),
    name = "",
    guide = guide_colorbar(reverse = FALSE)
  ) +
  scale_fill_gradient(
    # midpoint = mean(range(d_ng_measurability_importance$m_group)),
    limits = c(0,1),
    low = "darkslateblue",
    # mid = "darkturquoise",
    high = "aquamarine"
  ) +
  theme_minimal() +
  ylab("Node group") +
  xlab("Node groups combined Centrality (Pagerank)") +
  # geom_rug(aes(colour = m_dist_to_trace), alpha = 0.6, sides = "b") +

```

```

# coord_cartesian(clip = "off") +
guides(fill = "none") +
theme(legend.position = "bottom",
      legend.justification = "center",
      legend.box.just = "center",
      legend.text = element_text(size = 8),
      legend.key.width = unit(1, "cm"),
      axis.ticks.y = element_blank(),
      axis.line.y = element_blank(),
      panel.grid.major.y = element_blank(),
      panel.grid.minor.y = element_blank())

# p_top_node_pagerank_no_outcome_measurability

# extract legend
leg <- get_legend(p_top_node_pagerank_no_outcome_measurability)

# arrange with relative heights (legend gets a small slice)
p_node_importance_with_measurability <- (wrap_elements(leg) /
  (p_top_node_pagerank_no_outcome_measurability + theme(legend.position = "none"))) +
  plot_layout(heights = c(0.2, 1))

# p_node_importance_with_measurability
# ggsave("figures/FIG--node_pagerank_no_outcome_measurability.png", p_node_importance_with_m
#       width = 5,
#       height = 4,
#       dpi = 900)

```

With discrete scale instead

```

m_pal <- c("darkslateblue",
          "dodgerblue", "orange", "deeppink")

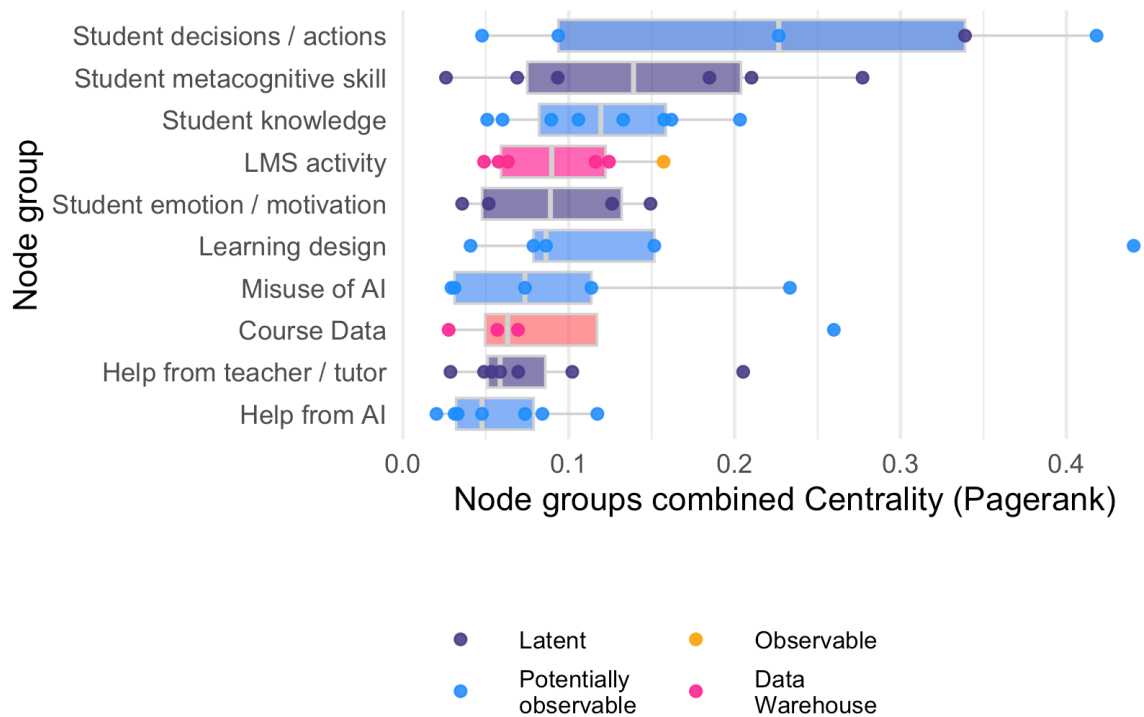
p_top_node_pagerank_no_outcome_measurability_discrete <-
  d_ng_measurability_importance |>
  mutate(
    measurability = case_when(
      measurability < (1/6) ~ "Latent",
      measurability < 0.5 ~ "Potentially observable",
      measurability < (5/6) ~ "Observable",
      TRUE ~ "Data warehouse") |>

```

```

        factor(levels = c("Latent", "Potentially observable", "Observable", "Data warehouse"))
    ) |>
    arrange(Model, node_group) |>
    ggplot(aes(x = pagerank_no_outcome, y = node_group)) +
    geom_boxplot(aes(fill = m_group), colour = "lightgray", size = 0.4, alpha = 0.7) +
    # geom_line(aes(x = pagerank_no_outcome, y = node_group, group = Model), alpha = 0.6) +
    geom_point(aes(colour = measurability), alpha = 0.9) +
    scale_color_manual(values = m_pal, # or retro four
                      labels = c("Latent", "Potentially\nobservable", "Observable", "Data\nwarehouse"),
                      name = "")
  ) +
  scale_fill_gradientn(
    colours = m_pal,
    breaks = c(0, 1/3, 2/3, 5/6)
  ) +
  theme_minimal() +
  ylab("Node group") +
  xlab("Node groups combined Centrality (Pagerank)") +
  # geom_rug(aes(colour = m_dist_to_trace), alpha = 0.6, sides = "b") +
  # coord_cartesian(clip = "off") +
  guides(fill = "none",
         color = guide_legend(nrow = 2, byrow = FALSE)) +
  theme(legend.position = "bottom",
        legend.direction = "vertical",
        legend.justification = "left",
        legend.text = element_text(size = 8),
        legend.key.width = unit(1, "cm"),
        axis.ticks.y = element_blank(),
        axis.line.y = element_blank(),
        panel.grid.major.y = element_blank(),
        panel.grid.minor.y = element_blank())
p_top_node_pagerank_no_outcome_measurability_discrete

```



```
#
# # extract legend
# leg <- get_legend(p_top_node_pagerank_no_outcome_measurability_discrete)
#
# # arrange with relative heights (legend gets a small slice)
# p_node_importance_with_measurability <- (wrap_elements(leg) /
#   (p_top_node_pagerank_no_outcome_measurability_discrete + theme(legend.position = "none")
#   plot_layout(heights = c(0.2, 1))

ggsave("figures/FIG--node_pagerank_no_outcome_measurability.jpg",
  p_top_node_pagerank_no_outcome_measurability_discrete,
  width = 5,
  height = 4,
  dpi = 900)
```

As table

```
all_nodes_with_metrics |>
  distinct(name, measurability) |>
```

```
janitor::tabyl(measurability) |>
janitor::adorn_totals("row")
```

```
measurability  n    percent
      DW   5 0.06756757
      O   9 0.12162162
      PO  31 0.41891892
      L  29 0.39189189
    Total 74 1.00000000
```

```
# Pagerank no outcome - dist to traced version - old
```

```
d_ng_trace_importance <-
  all_node_groups_aggregated_metrics |>
  inner_join(all_node_groups_distance_to_traced, by = c("Model", "node_group")) |>
  filter(node_group != "Grade.Outcome") |>
  inner_join(node_group_nice_name, by = "node_group") |>
  mutate(node_group = nice_name) |>
  group_by(node_group) |>
  mutate(p_traced_group = mean(m_dist_to_trace)) |>
  ungroup() |>
  mutate(node_group = fct_reorder(node_group, pagerank_no_outcome)) |>
  arrange(desc(node_group)) |>
  filter(n > 3) # need at least half the models with node group
```

```
p_top_node_pagerank_no_outcome_trace_dist <-
  d_ng_trace_importance |>
  # arrange(Model, node_group) |>
  ggplot(aes(x = pagerank_no_outcome, y = node_group)) +
  geom_boxplot(aes(fill = p_traced_group), colour = "lightgray", size = 0.4, alpha = 0.7) +
  # geom_line(aes(x = pagerank_no_outcome, y = node_group, group = Model), alpha = 0.6) +
  geom_point(aes(colour = m_dist_to_trace), alpha = 0.9) +
  scale_color_gradient2(
    low = "darkslateblue",
    mid = "coral",
    high = "deeppink",
    midpoint = mean(range(d_ng_trace_importance$m_dist_to_trace)),
    breaks = range(d_ng_trace_importance$m_dist_to_trace),
    labels = c("Observed", "Latent"),
    name = "",
```

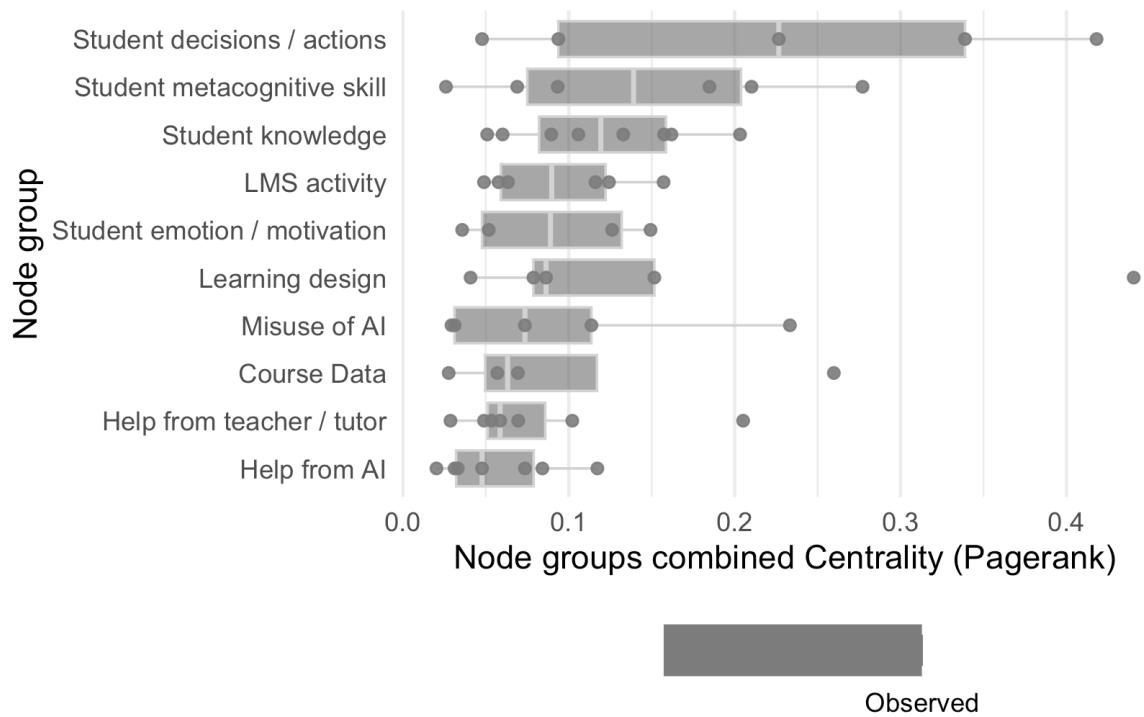


```

    guide = guide_colorbar(reverse = TRUE)
  ) +
  scale_fill_gradient2(
    midpoint = mean(range(d_ng_trace_importance$m_dist_to_trace)),
    low = "darkslateblue",
    mid = "coral",
    high = "deeppink"
  ) +
  theme_minimal() +
  ylab("Node group") +
  xlab("Node groups combined Centrality (Pagerank)") +
  # geom_rug(aes(colour = m_dist_to_trace), alpha = 0.6, sides = "b") +
  # coord_cartesian(clip = "off") +
  guides(fill = "none") +
  theme(legend.position = "bottom",
        axis.ticks.y = element_blank(),
        axis.line.y = element_blank(),
        panel.grid.major.y = element_blank(),
        panel.grid.minor.y = element_blank())

p_top_node_pagerank_no_outcome_trace_dist

```



```
ggsave("figures/FIG--node_pagerank_no_outcome_trace_distance.jpg", p_top_node_pagerank_no_outcome_trace_distance,
  width = 5,
  height = 4,
  dpi = 900)

d_ng_trace_importance |>
  group_by(node_group) |>
  summarise(m_dist_to_trace = mean(m_dist_to_trace)) |>
  arrange(desc(m_dist_to_trace))
```

```
# A tibble: 10 x 2
  node_group m_dist_to_trace
  <fct>      <dbl>
1 Course Data Inf
2 LMS activity Inf
3 Student emotion / motivation 1.96
4 Misuse of AI 1.8
5 Help from teacher / tutor 1.79
6 Learning design 1.67
7 Student knowledge 1.62
```

8 Help from AI	1.57
9 Student decisions / actions	1.43
10 Student metacognitive skill	1.33

Above plot but lines joining models that are like?

Table: Node lists, with types, groups and importance metrics

```
# could add distance to traced
node_groups_table <-
  all_nodes_with_metrics |>
  inner_join(name_node_nice_name, by = "name") |>
  mutate(name = if_else(
    node_group == "H.AI.good" & measurability == "Traced",
    str_c(nice_name, "*"),
    nice_name
  )) |>
  inner_join(node_group_nice_name |> rename(ng_nice_name = nice_name), by = "node_group")
  mutate(node_group = ng_nice_name) |>
  group_by(node_type, node_group) |>
  mutate(measurability = case_when(
    all(measurability == "Traced") ~ "Traced",
    all(measurability == "Untraced") ~ "Untraced",
    TRUE ~ str_c("Some traced* (", sum(measurability == "Traced"), " of ", n(), ")")
  )) |>
  group_by(node_type, node_group, measurability) |>
  summarise(
    nodes = str_c(unique(name), collapse = "; "),
    n = sum(node_w),
    pagerank = round(sum(pagerank_no_outcome) / 8, 2)
    # ,btw = round(sum(btw_rel), 3)
  ) |>
  mutate(pagerank = if_else(node_group == "Grade.Outcome", NA, pagerank)) |>
  # mutate(node_type = case_when(
  #   node_type == "L" ~ "Learning Design",
  #   node_type == "S" ~ "Student",
  #   node_type == "T" ~ "Teacher",
  #   node_type == "C" ~ "Course Data",
  #   node_type == "G" ~ "Grade Data",
  #   node_type == "H" ~ "Help"
  # )) |>
  arrange(desc(n), desc(pagerank)) |>
```

```
select(node_group, node_type, nodes, n, measurability, pagerank) |>
ungroup()
```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by node_type, node_group, and measurability.
- i Output is grouped by node_type and node_group.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(node\_type, node\_group, measurability))` for per-operation grouping (`?dplyr::dplyr\_by`) instead.

```
node_groups_table
```

```
# A tibble: 21 x 6
```

	node_group <chr>	node_type <fct>	nodes <chr>	n <dbl>	measurability <chr>	pagerank <dbl>
1	Learning design	Learning des~	Task~	11	Some traced*~	0.1
2	Help from AI	Help	Help~	11	Some traced*~	0.05
3	Student decisions / actions	Student	Stud~	10	Some traced*~	0.14
4	Student knowledge	Student	Stud~	10	Some traced*~	0.12
5	Student metacognitive skill	Student	Stud~	9	Some traced*~	0.11
6	Help from teacher / tutor	Help	Help~	9	Some traced*~	0.07
7	Student emotion / motivation	Student	Stud~	9	Some traced*~	0.05
8	LMS activity	Learning data	LMS ~	8	Some traced*~	0.07
9	Assessment grade	Assessment d~	Asse~	8	Some traced*~	0
10	Preliminary assessments	Assessment d~	Mark~	7	Some traced*~	0.08

```
# i 11 more rows
```

```
all_nodes_with_metrics |>
  distinct(name) |>
  nrow() |>
  paste("distinct nodes")
```

```
[1] "74 distinct nodes"
```

```
all_nodes_with_metrics |>
  distinct(node_group) |>
  nrow() |>
  paste("node groups")
```

```
[1] "21 node groups"
```

```
all_nodes_with_metrics |>
  distinct(name, measurability) |> count(measurability)
```

```
# A tibble: 4 x 2
  measurability     n
  <fct>         <int>
1 DW             5
2 O              9
3 P0            31
4 L            29
```

```
tb1 <- node_groups_table |> filter(n >= 7) |> arrange(desc(n))
tb2 <- node_groups_table |> filter(n < 7) |> arrange(desc(n))
tb1 |> clipr::write_clip()
tb2 |> clipr::write_clip()
```

What about grouping by the node category first, so it's hierarchical?

```
node_types_table <-
  node_groups_table |>
  group_by(node_type) |>
  mutate(nt = sum(n)) |>
  ungroup() |>
  arrange(desc(nt), desc(n)) |>
  select(node_type, nt, node_group, nodes, n, measurability, pagerank) |>
  mutate(node_type = str_c(node_type, " (", nt, ")")) |>
  select(-nt)

node_types_table
```

```
# A tibble: 21 x 6
  node_type    node_group          nodes      n measurability pagerank
  <chr>        <chr>          <chr> <dbl> <chr>          <dbl>
1 Student (44) Student decisions / actions Stude~    10 Some traced*~    0.14
2 Student (44) Student knowledge       Stude~    10 Some traced*~    0.12
3 Student (44) Student metacognitive skill Stude~     9 Some traced*~    0.11
4 Student (44) Student emotion / motivation Stude~     9 Some traced*~    0.05
5 Student (44) Student attributes       Stude~     4 Some traced*~    0.02
6 Student (44) Student prior knowledge   Stude~     2 Some traced*~    0.01
7 Help (30)    Help from AI              Help ~    11 Some traced*~    0.05
```

```

8 Help (30)      Help from teacher / tutor      Help ~      9 Some traced*~      0.07
9 Help (30)      Misuse of AI                    Misus~      6 Some traced*~      0.06
10 Help (30)     Help from peers                  Help ~      3 Some traced*~      0.01
# i 11 more rows

```

```

tbt1 <- node_types_table |> filter(str_detect(node_type, "Student|Help"))
tbt2 <- node_types_table |> filter(!str_detect(node_type, "Student|Help"))
tbt1 |> clipr::write_clip()
tbt2 |> clipr::write_clip()

```

Edge / path importance

```

all_edge_group_with_metrics <-
  edges_with_metrics |>
  mutate(from = code_node_group(from),
         to = code_node_group(to)) |>
  group_by(from, to) |>
  summarise(
    n = sum(n),
    max_n = sum(max_n),
    edge_p_w = mean(edge_p_w),
    p_aligned = mean(switched == "aligned")
  )

```

`summarise()` has regrouped the output.

- i Summaries were computed grouped by from and to.
- i Output is grouped by from.
- i Use `summarise(.groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(from, to))` for per-operation grouping
(`?dplyr::dplyr\_by`) instead.

```

edge_group_analysis_edgelist <- all_edge_group_with_metrics |>
  mutate(edge_w = n + edge_p_w * n) |>
  arrange(desc(edge_w))

```

```

# the node group one here isn't one row per node group, and doesn't have the node type!
edge_group_analysis_graph <- tbl_graph(
  nodes = all_node_groups_aggregated_metrics |>
    rename(name = node_group) |>

```

```

group_by(name, node_type) |>
summarise(
  p_traced = mean(p_traced),
  pgrnk = mean(pagerank_no_outcome),
  node_w = sum(n)
) ,
edges = edge_group_analysis_edgelist)

```

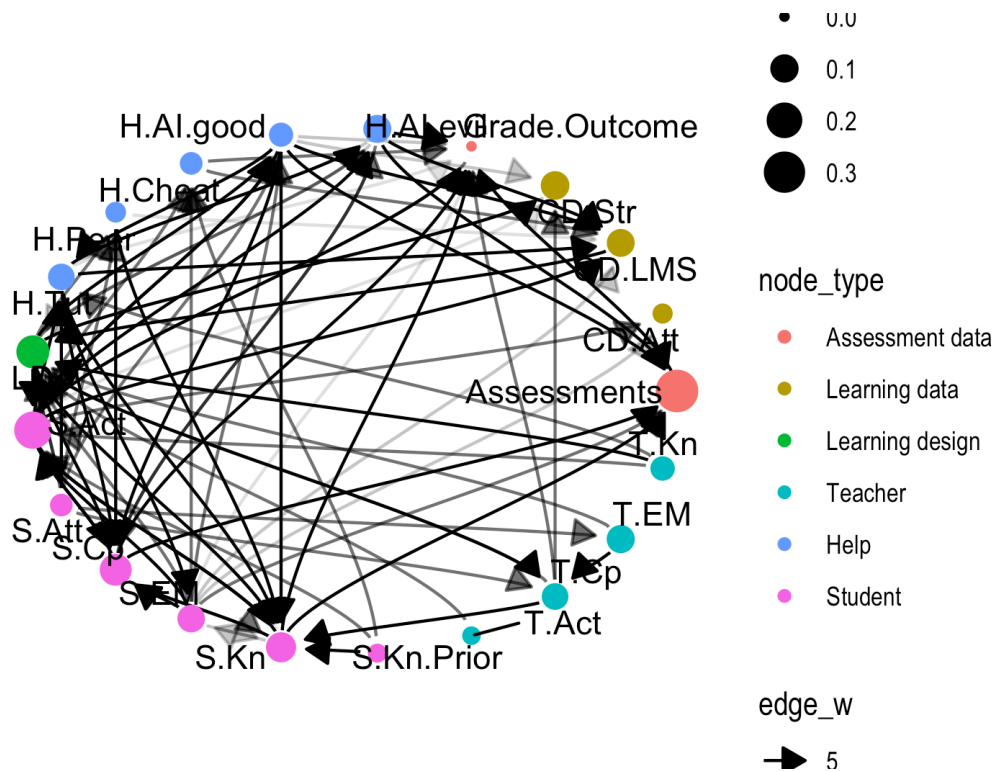
`summarise()` has regrouped the output.

- i Summaries were computed grouped by name and node_type.
- i Output is grouped by name.
- i Use `summarise(groups = "drop\_last")` to silence this message.
- i Use `summarise(.by = c(name, node\_type))` for per-operation grouping (`?dplyr::dplyr_by`) instead.

```

edge_group_analysis_graph %N>%
  arrange(name) |>
  # activate(edges) |>
  # filter(edge_w > 8) |>
  ggraph(
    layout = "circle"
    # layout = "sugiyama"
  ) +
  geom_edge_diagonal(
    aes(alpha = edge_w),
    strength = 0.5,
    arrow = arrow(length = unit(3, 'mm'),
                  type = "closed"),
    start_cap = circle(2, 'mm'),
    end_cap = circle(3, 'mm')) +
  geom_node_point(aes(size = pgrnk, color = node_type)) +
  geom_node_text(aes(label = name), repel = TRUE) +
  scale_edge_alpha(range = c(0.1,9)) +
  theme_graph()

```



Path importance - what about the sets of implied conditional independencies? Is there a way to look at the sets of these and see if any contradict each other??

Simulation

Now we examine the feasibility of learning these models from data. A typical counter to going to the effort of eliciting these models from domain experts is that they can be learned from the data alone. However, we have seen here that much of the data is not present in the institutions ecosystem, with the nodes being deemed inherently latent (35%), potentially observable (42.3%), observable but not currently in the data (7.3%), or already measured and stored in the current data ecosystem (15.4%).

We design this experiment as follows. Each supplied model is used to generate synthetic data, assuming each node has an equal effect on its descendants. HOW WILL THIS DO LOOPS? For the “potentially observable” nodes a single descendant node is added, a proxy measurement node that is not the exact node measured but whose value depends only on the potentially observable node and some additional noise. The set of proxy nodes for the potentially observable (PO) nodes we denote ($\hat{PO}$). We choose three scenarios of data quality: idyllic, typical, and noisy. Each of these scenarios is represented by the ratio of the path effect to the noise.


```

sim_signal_strength <- list("idyllic" = 0.9, "typical" = 0.4, "noisy" = 0.1)

tibble(Scenario = names(sim_signal_strength),
      Beta = as.numeric(sim_signal_strength)) |>
  mutate(
    `Variance due to each parent node` = Beta^2 / (1 + Beta^2)
  ) |>
  knitr::kable(digits = 3)

```

Scenario	Beta	Variance due to each parent node
idyllic	0.9	0.448
typical	0.4	0.138
noisy	0.1	0.010

Here we see that in a “typical” scenario we assume that each path coefficient is 0.4.

We then apply a structure learning algorithm to each models simulated data in three phases: learning from the currently measured data only (only DW nodes), learning from all observable nodes (DW and O) nodes, and finally learning from all potentially available data (DW, O and $\hat{PO}$). This represents increasing levels of resourcing applied to learning the underlying causal structure, given the model.

Note we are working under a local assumption that each model is the ‘ground truth’, but fully acknowledge that we do not know for this system what the true model is. However, if the structure learning claims of application to discover latent nodes are to be believed (CITE) then we should see some reasonable convergence towards each locally true model.

```

# maybe use dagitty::simulateSEM() or dagitty::simulateLogistic()
# dagitty suggests using lavaan instead for SEM

# should add layout!!
# use same x, y from main graph

m_build_model_with_proxy_nodes <- function(
  m = "S1",
  x_p_nudge = 0, y_p_nudge = -2,
  S.Kn_to_Obs = TRUE) {
  model <- m
  if (S.Kn_to_Obs) { # for running sims
    all_nodes_adj <-
      all_nodes_with_metrics |>

```

```

      mutate(measurability = if_else(
        # just picking S.Kn.Tech for S7
        name %in% c("S.Kn", "S.Kn.Tech"), "0",
        measurability
      ) |>
        factor(levels = levels(all_nodes_with_metrics$measurability)))
} else {
  all_nodes_adj <- all_nodes_with_metrics
}
nodes <- all_nodes_adj |>
  inner_join(merged_all_graph_layout_adjusted |>
    select(name, x, y),
    by = "name") |>
  filter(Model == toupper(model)) |>
  select(name, x, y, node_group, node_type, measurability) |>
  bind_rows( # adding proxy nodes for P0 nodes
    all_nodes_adj |>
      inner_join(merged_all_graph_layout_adjusted |>
        select(name, x, y),
        by = "name") |>
      filter(Model == model, measurability == "P0") |>
      select(name, node_type, x, y) |>
      mutate(name = str_c(name, "_p"),
        # Need to place proxy nodes close - might cause issues
        x = x + x_p_nudge,
        y = y + y_p_nudge
      ) |>
      mutate(node_group = "Proxy") |>
      mutate(measurability = "0") # the assumption for the proxy
    ) |>
  mutate(measurability = factor(
    measurability,
    levels = levels(all_nodes_with_metrics$measurability)))

edges <- all_edges |>
  filter(Model == tolower(model)) |>
  select(-Model) |>
  bind_rows(
    tibble(
      from = nodes |>
        filter(measurability == "P0") |>
        pull(name)) |>

```

```

      mutate(to = str_c(from, "_p"))
    )

new_tidydag <- tidygraph::tbl_graph(
  nodes = nodes, edges = edges, directed = TRUE
)
new_dag <- dagitty_from_edge_df(edges)

# setting coordinates of dagitty object - needs to be a named list
node_names <- nodes$name
node_coord_list <- list(x = nodes$x, y = nodes$y)
names(node_coord_list$x) <- node_names
names(node_coord_list$y) <- node_names
coordinates(new_dag) <- node_coord_list

node_list_dw <- nodes |> filter(measurability == "DW") |> pull(name)
node_list_dwo <- nodes |>
  filter(!str_detect(name, "_p")) |>
  filter(measurability %in% c("DW", "O")) |>
  pull(name)
node_list_dwoop <- nodes |>
  filter(measurability %in% c("DW", "O")) |>
  pull(name)

return(list(
  dag = new_dag, # new dag in dagitty format
  tdag = new_tidydag, # new dag as tidydag
  node_list_dw = node_list_dw, # list of nodes in DW
  node_list_dwo = node_list_dwo, # list of nodes in DW or Observable
  node_list_dwoop = node_list_dwoop # list of all non-latent nodes
))
}

simSEM <- partial(simulateSEM, eps = 1, standardized = F, N = 1000)

# d.noisy <- simSEM(new_dag, b.default = sim_signal_strength[["noisy"]])
# d.typical <- simSEM(new_dag, b.default = sim_signal_strength[["typical"]])
# d.ideal <- simSEM(new_dag, b.default = sim_signal_strength[["ideal"]])

bn_fit_model_list <- function(m_proxy_list, Beta = 0.6) {

```

```

# getting simulated data, and subsets
d.sim.sem <- simSEM(m_proxy_list$dag, b.default = Beta)
d.dw <- as.data.frame(d.sim.sem[,m_proxy_list$node_list_dw])
d.dwo <- as.data.frame(d.sim.sem[,m_proxy_list$node_list_dwo])
d.dwoop <- as.data.frame(d.sim.sem[,m_proxy_list$node_list_dwoop])

# fitting BNets
bn.full = bnlearn::bn.fit(bnlearn::hc(d.sim.sem), d.sim.sem)
bn.dw = bnlearn::bn.fit(bnlearn::hc(d.dw), d.dw)
bn.dwo = bnlearn::bn.fit(bnlearn::hc(d.dwo), d.dwo)
bn.dwoop = bnlearn::bn.fit(bnlearn::hc(d.dwoop), d.dwoop)

return( # returns a named list of simmed data and models
  list(
    # The full data set
    d.full = d.sim.sem,
    bn.full = bn.full,
    # Only nodes in the Data Warehouse (DW)
    d.dw = d.dw,
    bn.dw = bn.dw,
    # DW and Observable (O) nodes
    d.dwo = d.dwo,
    bn.dwo = bn.dwo,
    # Including Potentially Observable (PO), so ALL non-latent nodes
    d.dwoop = d.dwoop,
    bn.dwoop = bn.dwoop
  ))
}

bn_to_dagitty <- function(bn) {
  edges <- bnlearn::arcs(bn) |>
  as_tibble() |>
  mutate(edge_str = str_c(from, " -> ", to))

  dagitty(str_c(
    "dag{",
    str_c(edges$edge_str, collapse = "; "),
    "}"
  ))
}

# Thinking of a simple dag X -> Y, we have

```

```

# Var(Y) = b^2 * Var(X) + Var(eps) -- eps is the noise, and here it is set to 1 by default
# then Var(Y) = b^2 + 1
# Since the R^2 is the ratio of the variance explained by X:
# R^2 = Var(explained) / Var(total)
# R^2 = b^2 / (b^2 + 1)

dagitty_to_bn <- function(dag) {

  edges <- dagitty::edges(dag) |>
    as_tibble() |>
    filter(e == "->") |> # directed edges only, excludes bidirected
    select(v, w) |>
    rename(from = v, to = w)

  nodes <- unique(c(edges$from, edges$to))

  # Build empty network then set arcs
  net <- bnlearn::empty.graph(nodes)
  bnlearn::arcs(net) <- as.matrix(edges)
  net
}

causal_estimate_over_all_adj_sets <- function(
  dag, data,
  exposure = "S.Kn", outcome = "Grade",
  intervention_val = 1) {
  # This estimates P(Grade|S.Kn, Adjustment Set) over all adjustment sets
  # This will give a range of that the model suggests is the true effect
  adj_sets <- adjustmentSets(dag, exposure = exposure, outcome = outcome,
    type = "all")

  # Throw error if none!
  estimate_per_set <- map(adj_sets, function(adj_set) {
    if (length(adj_set) == 0) {
      formula_str <- str_c(outcome, " ~ ", exposure)
    } else {
      formula_str <- str_c(outcome, " ~ ", exposure, " + ",
        str_c(adj_set, collapse = " + "))
    }
    fit <- lm(as.formula(formula_str), data = data)
    preds <- predict(fit,
      newdata = data |>

```

```

      mutate({{ exposure }} := intervention_val),
      se.fit = TRUE)

list(
  mean = mean(preds$fit),
  se    = sqrt(mean(preds$se.fit^2))
)
}
)

estimates <- map_dbl(estimate_per_set, "mean")
std_errors <- map_dbl(estimate_per_set, "se")
return(list(adjustment_set = adj_sets,
            estimate = estimates,
            std_error = std_errors))
}

generate_causal_estimate_comparisons <- function(
  Model = "S1",
  Beta = 0.6, # strength of signal along paths
  # might need to change for different models depending on present nodes
  exposure = "S.Kn", outcome = "Grade",
  intervention_val = 1,
  # for placing proxy nodes
  x_p_nudge = 0, y_p_nudge = -2
) {
  ground_truth_dag <- ddags[[Model]]
  model_list <- m_build_model_with_proxy_nodes(
    m = Model,
    x_p_nudge = x_p_nudge,
    y_p_nudge = y_p_nudge)

  bn_list <- bn_fit_model_list(model_list, Beta = Beta)
  # Ground truth uses full simmed data
  est <- partial(causal_estimate_over_all_adj_sets,
                exposure = exposure, outcome = outcome,
                intervention_val = intervention_val)
  ground_truth <- est(dag = ground_truth_dag, data = bn_list$d.full)
  can_estimate_dw <-
    (exposure %in% model_list$node_list_dw) & (outcome %in% model_list$node_list_dw)
  can_estimate_dwo <-
    (exposure %in% model_list$node_list_dwo) & (outcome %in% model_list$node_list_dwo)
  can_estimate_dwoop <-

```

```

      (exposure %in% model_list$node_list_dwoop) & (outcome %in% model_list$node_list_dwoop)
    if (can_estimate_dw) {
      dw <- est(
        dag = bn_to_dagitty(bn_list$bn.dw) ,
        data = bn_list$d.dw)
    } else {
      dw <- list("adjustment_set" = NA, "estimate" = NA)
      message("Nodes not found in DW node list")
    }

    if (can_estimate_dwo) {
      dwo <- est(
        dag = bn_to_dagitty(bn_list$bn.dwo) ,
        data = bn_list$d.dwo)
    } else {
      dwo <- list("adjustment_set" = NA, "estimate" = NA)
      message("Nodes not found in DWO node list")
    }

    if (can_estimate_dwoop) {
      dwoop <- est(
        dag = bn_to_dagitty(bn_list$bn.dwoop) ,
        data = bn_list$d.dwoop)
    } else {
      dwoop <- list("adjustment_set" = NA, "estimate" = NA)
      message("Nodes not found in DWOOP node list")
    }

    estimates <- tibble(
      Model = Model,
      DAG = c("Ground Truth", "Data Warehouse", "Observable", "Potentially Observable") |>
        factor(levels = c("Ground Truth", "Data Warehouse", "Observable", "Potentially Observable")),
      exposure = exposure,
      outcome = outcome,
      adjustment_sets = list(
        ground_truth$adjustment_set,
        dw$adjustment_set,
        dwo$adjustment_set,
        dwoop$adjustment_set
      ),
      estimate_mean = c(
        mean(ground_truth$estimate),

```

```

        mean(dw$estimate),
        mean(dwo$estimate),
        mean(dwoop$estimate)
    ),
    estimate_std_errors = c(
        mean(ground_truth$std_error),
        mean(dw$std_error),
        mean(dwo$std_error),
        mean(dwoop$std_error)
    ),
    estimate_sd = c(
        sd(ground_truth$estimate),
        sd(dw$estimate),
        sd(dwo$estimate),
        sd(dwoop$estimate)
    )
)

return( # Monster list of everything we need
    list(
        Model = Model,
        DAG = ground_truth_dag,
        augmented_DAGs = model_list,
        fitted_BN = bn_list,
        estimates = estimates
    )
)
}

s1_estimates <-
  generate_causal_estimate_comparisons(
    Model = "S1",
    Beta = sim_signal_strength[["typical"]])

```

Nodes not found in DW node list

Warning in mean.default(dw\$std_error): argument is not numeric or logical:
returning NA

```

s2_estimates <-
  generate_causal_estimate_comparisons(

```



```
Model = "S2",  
Beta = sim_signal_strength[["typical"]])
```

Nodes not found in DW node list

Warning in mean.default(dw\$std_error): argument is not numeric or logical:
returning NA

```
# s3_estimates <- # ERROR! This one is cycle, and no adj sets  
#   generate_causal_estimate_comparisons(  
#     Model = "S3",  
#     Beta = sim_signal_strength[["typical"]])  
s4_estimates <- # Acyclic,  
  generate_causal_estimate_comparisons(  
    Model = "S4",  
    Beta = sim_signal_strength[["typical"]])
```

Nodes not found in DW node list

Warning in mean.default(dw\$std_error): argument is not numeric or logical:
returning NA

```
s5_estimates <-  
  generate_causal_estimate_comparisons(  
    Model = "S5",  
    Beta = sim_signal_strength[["typical"]])
```

Nodes not found in DW node list

Warning in mean.default(dw\$std_error): argument is not numeric or logical:
returning NA

```
# s6_estimates <- # cyclic, no adj sets  
#   generate_causal_estimate_comparisons(  
#     Model = "S6",  
#     Beta = sim_signal_strength[["typical"]])  
# s7_estimates <- # Different exposure - not sure what to do  
#   generate_causal_estimate_comparisons(  
#     Model = "S7",
```

```
#      exposure = "S.Kn.Tech",
#      Beta = sim_signal_strength[["typical"]])
s8_estimates <-
  generate_causal_estimate_comparisons(
    Model = "S8",
    Beta = sim_signal_strength[["typical"]])
```

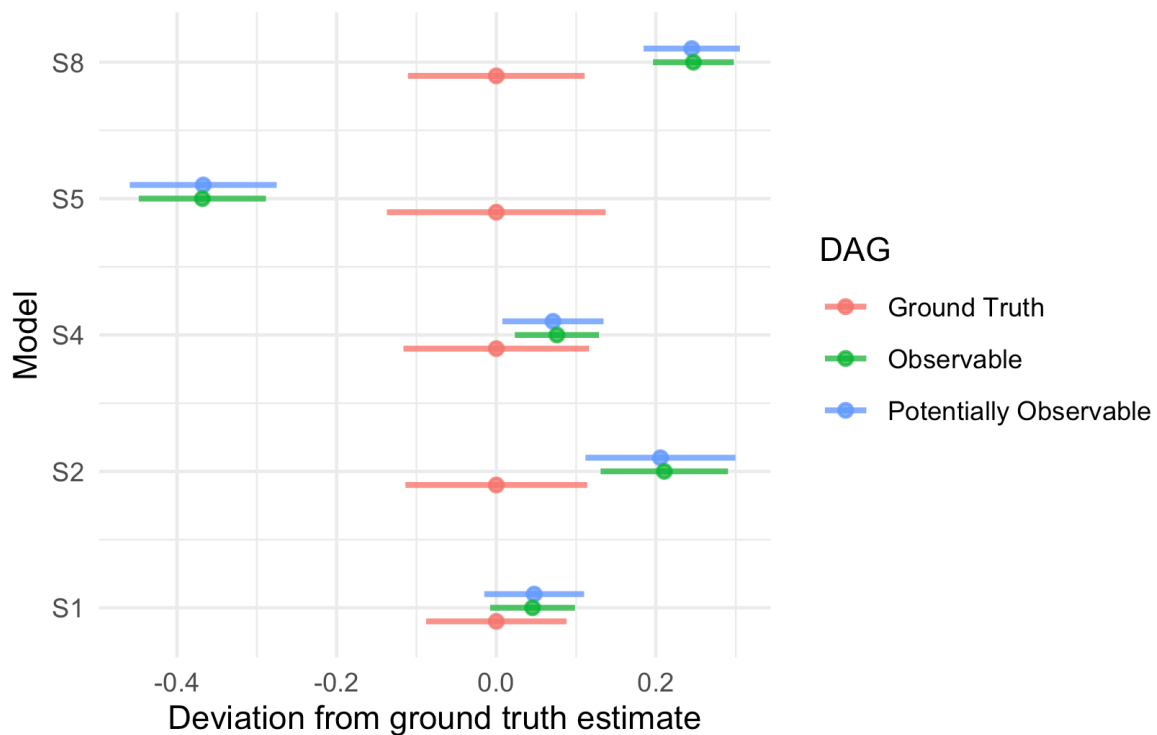
Nodes not found in DW node list

Warning in mean.default(dw\$std_error): argument is not numeric or logical:
returning NA

```
all_estimates <-
  s1_estimates$estimates |>
  bind_rows(s2_estimates$estimates) |>
  # bind_rows(s3_estimates$estimates) |>
  bind_rows(s4_estimates$estimates) |>
  bind_rows(s5_estimates$estimates) |>
  # bind_rows(s6_estimates$estimates) |>
  # bind_rows(s7_estimates$estimates) |>
  bind_rows(s8_estimates$estimates) |>
  filter(!is.na(estimate_mean)) |>
  group_by(Model) |>
  mutate(
    gt_est_diff = estimate_mean - estimate_mean[DAG == "Ground Truth"] )|>
  ungroup() |>
  mutate(error = estimate_std_errors + replace_na(estimate_sd, 0)) |>
  select(Model, DAG, estimate_mean, error, gt_est_diff, everything())

dodge_amt <- 0.1
all_estimates |>
  mutate(
    y_pos = as.numeric(factor(Model)) +
      (as.numeric(factor(DAG)) - mean(as.numeric(factor(DAG)))) * dodge_amt
  ) |>
  ggplot(aes(color = DAG, y = y_pos)) +
  # geom_pointinterval(
  #   aes(x = gt_est_diff,
  #       xmin = gt_est_diff - 2 * error,
  #       xmax = gt_est_diff + 2 * error),
  #   alpha = 0.3, size = 1
```

```
# ) +
geom_pointinterval(
  aes(x = gt_est_diff,
      xmin = gt_est_diff - 1 * error,
      xmax = gt_est_diff + 1 * error),
  alpha = 0.8, size = 3
) +
scale_y_continuous(
  breaks = seq_along(levels(factor(all_estimates$Model))),
  labels = levels(factor(all_estimates$Model)),
  name = "Model"
) +
xlab("Deviation from ground truth estimate") +
theme_minimal()
```

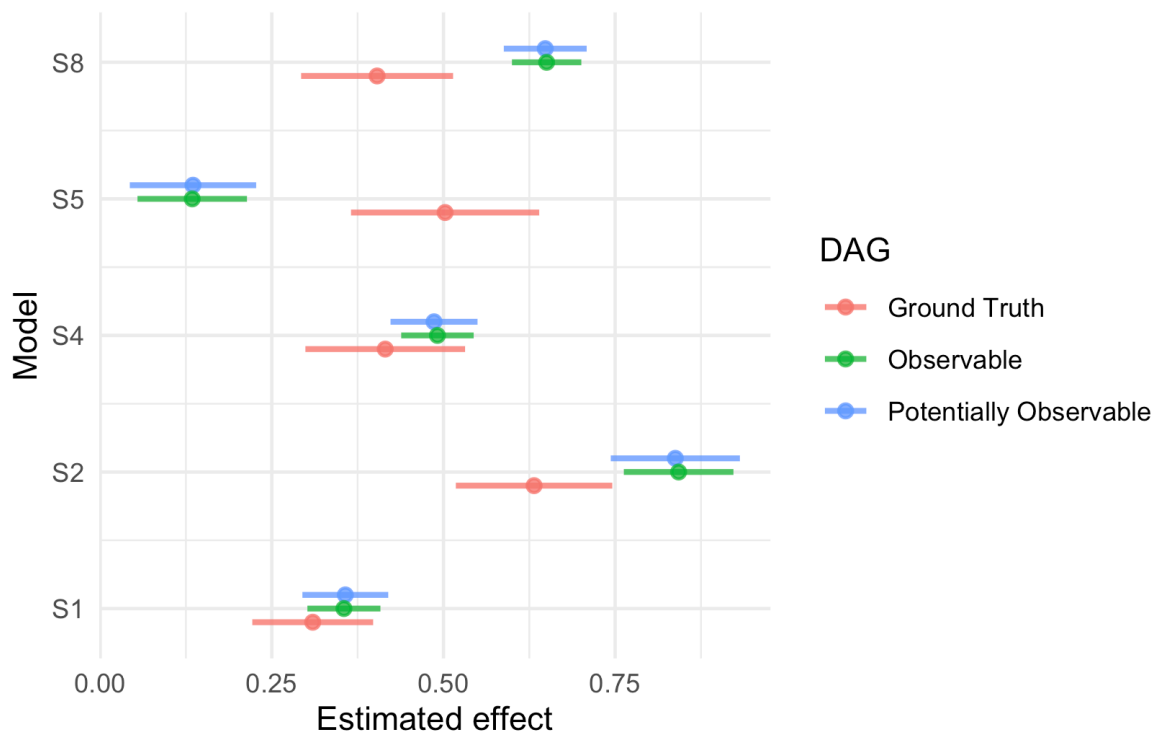


```
all_estimates |>
  mutate(
    y_pos = as.numeric(factor(Model)) +
      (as.numeric(factor(DAG)) - mean(as.numeric(factor(DAG)))) * dodge_amt
  ) |>
```

```

ggplot(aes(color = DAG, y = y_pos)) +
  geom_pointinterval(
    aes(x = estimate_mean,
        xmin = estimate_mean - 1 * error,
        xmax = estimate_mean + 1 * error),
    alpha = 0.8, size = 3
  ) +
  scale_y_continuous(
    breaks = seq_along(levels(factor(all_estimates$Model))),
    labels = levels(factor(all_estimates$Model)),
    name = "Model"
  ) +
  xlab("Estimated effect") +
  theme_minimal()

```



```

# keep working through the ?bnlearn helper. The causal query might be a good metric ?cpquery

# shared_nodes <- intersect(
#   bnlearn::nodes(bn.full),

```

```

# bnlearn::nodes(bn.dwoop)
# )

#
# library(bnlearn)
# # Query both - e.g. probability Grade is above some threshold
# # given an intervention on an observable node
# # I also want to query the original DAG. How well do these represent that
# # Ground truth from simulation
# d.typical |>
#   filter(CD > 1) |>
#   summarise(p_truth = mean(Grade > 1))
#
# cpquery(bn.full,
#         event = (Grade > 1),
#         evidence = (CD > 1),
#         method = "ls",
#         n = 1e6)
#
# cpquery(bn.full,
#         event = (Grade > 1),
#         evidence = function(x) (x[["CD"]] > 1),
#         method = "lw",
#         n = 10000)
#
# cpquery(bn.dwoop,
#         event = (Grade > 1),
#         evidence = (CD_p > 1),
#         method = "ls")

```

Workflow: - Use adjustmentSets to find valid adjustment sets for each exposure-outcome pair (will need to be in both models, or all, or some??) - Use the simulated data with those adjustment sets as your ground truth causal estimates - Compare cpquery results against those — the gap tells you how much the structure learning is distorting causal conclusions vs just conditional ones

Maybe the adjustment sets should be for: - Knowledge -> Grade - AI use -> Grade - AI use -> Knowledge

Or perhaps just the first, and we definitely look at AI use as one of the conditional probabilities? Seems closest to the paper problem.

Ok, how about: - Estimate $P(\text{Grade}|\text{do}(\text{S.Kn}))$ in each: use the suggested adjustment sets. Maybe a Bayesian approach to get uncertainty (later if runtime allows)